选择题、填空题、简答题示例

1. 分析算法的T(n)和S(n)

分析@标注语句执行频度

```
(1) for(i=1; i<=n; i++)
        for(j=1; j<=i; j++)
            for(k=1; k<=j; k++)
                @ x+=delta;
```

解： 设m表示@执行次数，则有

$$m=\sum_{i=1}^{n}\sum_{j=1}^{i}\sum_{k=1}^{j}1=\sum_{i=1}^{n}\sum_{j=1}^{i}j=\sum_{i=1}^{n}\frac{i(i+1)}{2}$$

$$=\frac{1}{2}\left(\sum_{i=1}^{n}i^2+\sum_{i=1}^{n}i\right)=\frac{1}{2}\left[\frac{n(n+1)(2n+1)}{6}+\frac{n(n+1)}{2}\right]=\frac{n(n+1)(n+2)}{6}$$

1. 分析算法的T(n)和S(n)

(2) i=1; j=0;

while(i+j<=n) @ if(i>j) j++; else i++;

解： 每循环1次，i+j的值只增加1；第1次循环开始，i+j值为1，最后1次循环开始，i+j值为n，故@执行次数为n。

(3) x=n; y=0; while(x>=(y+1)*(y+1)) @ y++;

解： 令m=y+1，则 $m^2 \le n \Rightarrow m \le \sqrt{n} \Rightarrow m \le \lfloor \sqrt{n} \rfloor$

第1次循环开始，m=1；最后一次循环开始，$m = \lfloor \sqrt{n} \rfloor$ ，

每循环1次，m值增1，故@执行次数为$\lfloor \sqrt{n} \rfloor$。

1. 分析算法的T(n)和S(n)

续2完

分析时间复杂度及变量count的值

```
int Time(int n)  /* 已知n>2 */
{ count=0; x=2;
  while(x<n/2) { x*=2; count++; }
}
```

解：第1次循环开始，$x=2$；设最后一次循环开始，$x=2^k$。显然，函数调用结束时，count的值就是k。

$$2^k < n/2 \Leftrightarrow k < \log_2 n - 1$$

k应取满足上式的最大整数，即 $k = \lceil \log_2 n \rceil - 2$ 。

故时间复杂度为 $O(\log_2 n)$ 。

2. 堆栈与队列(特别是循环队列)

简答题示例：

若元素入栈次序为ABC，写出所有可能的出栈序列。

答：所有可能的出栈序列共5种，即

ABC	(PXPXPX)	P-入栈操作, X-出栈操作
ACB	(PXPPXX)	
BAC	(PPXXPX)	
BCA	(PPXPXX)	
CBA	(PPPXXX)	

2. 堆栈与队列(特别是循环队列)

续1

单项选择题示例：

(1) 长度为n且设有队头、队尾指针的链式队列中，出队操作的时间复杂度为__A__。

A. O(1)　B. O(n)　C. $O(n^2)$　D. $O(\log_2 n)$

(2) 已知元素进栈次序为a, b, c, d, e，经过合理的出入栈操作后，不能得到的出栈元素序列是__A__。

A. d, c, e, a, b　B. d, e, c, b, a　C. a, b, c, d, e　D. e, d, c, b, a

2. 堆栈与队列(特别是循环队列)

试说明以下算法的功能。

(1) Status algo1(Stack S)

```
{ int i, n, A[255];
  n=0;
  while(!StackEmpty(S)) { n++; Pop(S, A[n]); }
  for(i=1; i<=n; i++) Push(S, A[i]);
}
```

答：算法algo1的功能是实现堆栈S中元素逆序存储。

2. 堆栈与队列(特别是循环队列)


```
(2) Status algo2(Stack S, int e)
    { Stack T; int d;
      InitStack(T);
      while(!StackEmpty(S))
       { Pop(S, d); if(d!=e) Push(T, d);
       }
       while(!StackEmpty(T))
        { Pop(T, d); Push(S, d);
        }
    }
```

答：算法algo2的功能是删除堆栈S中值为e的元素。

2. 堆栈与队列(特别是循环队列)

试说明以下算法的功能。

```
Status algo3(Queue &Q)
  { Stack S; int d;
    InitStack(S);
    while(QueueEmpty(Q))
     { DeQueue(Q, d); Push(S, d);
     }
     while(!StackEmpty(S))
     { Pop(S, d); EnQueue(Q, d);
     }  }
```

答：算法algo3的功能是实现队列Q中元素逆序存储。

2. 堆栈与队列(特别是循环队列)

写出下面的递归算法，并消除递归。

$$F(n)=\begin{cases} n+1 & n=0 \\ nF(n/2) & n>0 \end{cases}$$

解：经分析，n应为整数，且$n/2$的值为整数商。

//递归算法

```
int F(int n)
{ if(n==0) return 1;
  return n*F(n/2);
}
```

//消除递归方法一：用数组

```
int F1(int n)
{ if(n<=1) return 1;
  int *a=new int[n/2+1]; int r;
  a[0]=1;
  for(i=1; i<=n/2; i++) a[i]=i*a[i/2];
  r=n*a[n/2];delete []a; return r; }
```

2. 堆栈与队列(特别是循环队列)

续5完

```
//消除递归方法二：用堆栈将递归算法非递归实现
int F2(int n)
{ if(n==0) return 1;
  Stack S; int r;  //r用于存储F(n/2)的值
  InitStack(S); k=n/2;
  while(k>0){Push(S, k); k=k/2;}
  r=1;  //r=F(0)
 while(!StackEmpty(S))
  { Pop(S, k); r=k*r;  //=左边r为F(k), =右边r为F(k/2)
  }
  return n*r;
}
```

F1(n)空间复杂度为$O(n/2)$;
F2(n)空间复杂度为$O(\log_2 n)$.

3. 前/中/后缀表达式的互化

填空题示例：

中缀表达式$a+(b-c)\times d$ 对应的前缀表达式是_____+ a * – b c d_____，

对应的后缀表达式是_____a b c - d * +_____。

后缀式3, 4, X, *, +, 2, Y, *, 5, /, - (表达式中逗号表示空格)对应的中缀式为_____(3+4*X)-(2*Y)/5_____。

4. 多维数组/半三角矩阵元素下标与顺序存储下标互算

若某高级语言30行×20列二维数组（设数组元素下标从0开始）元素按行序为主序存储，每个元素存储长度为8字节。若该数组首地址为0，则存储地址为600（该地址为十进制数）的元素的行下标=__3__，列下标=__15__。

某10阶对称方阵用一维数组按“行序为主序”顺序存储其下半三角元素（含主对角线元素），若所有下标从0开始，则行、列下标分别为6、7的矩阵元素在一维数组中的存储下标为__34__。

5. 广义表的表示；求表头和表尾

广义表(a,b,c,d)的表尾____D____。

A. b,c,d　　B. c,d　　C. d　　D. (b,c,d)

设有广义表L=((),()),则GetHead(L)=____( )____；GetTail(L)=____(( ))____。

6. 二叉树的前/中/后序以及层次遍历；各种计算问题

一棵深度为k且具有2^k-1个结点的二叉树称为__满二叉树__。这类二叉树的特点是，二叉树的每一层结点的个数都为__上一层的2倍__。

已知完全二叉树的第7层(第7层为最后一层)有10个叶结点，则整棵二叉树的结点为__73__。

4层完全二叉树至少有__8__个结点； 若完全二叉树中某结点编号为11，则该结点的右儿子编号为__23__，双亲结点编号为__5__。 (结点编号从1开始)

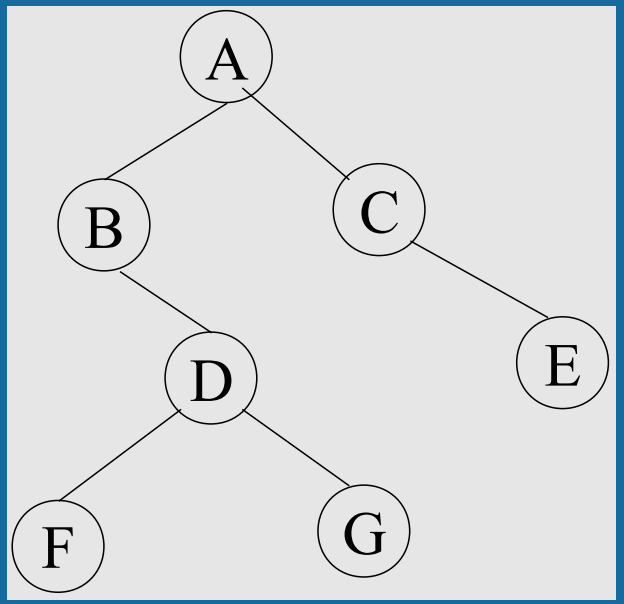
6. 二叉树的前/中/后序以及层次遍历；各种计算问题

有60个结点的完全二叉树中，叶子结点的数目为___30___。

一棵二叉树中，度为2的结点数为15，则叶子结点数为__B_____。

A. 12　　B. 16　　C. 18　　D. 20

二叉树如右图所示，写出先序、中序、后序遍历结点访问次序。



答：先序：ABDFGCE

　　中序：BFDGACE

　　后序：FGDBECA

设在一棵度数为3的树中，度数为3的结点数有2个，度数为2的结点数有1个，度数为1的结点数有2个，那么度数为0的结点数有（6）个。

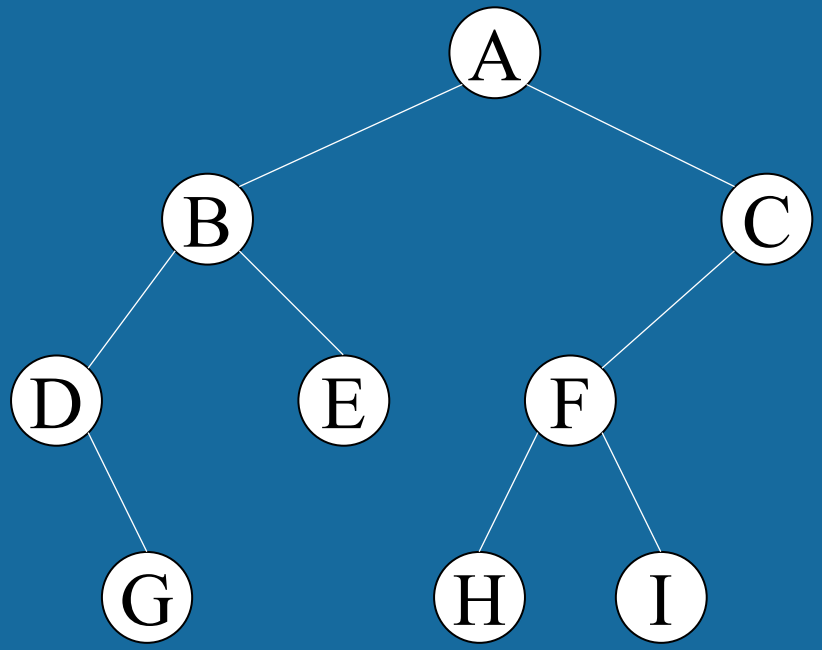
树中结点数等于所有结点度数的和加1，所以

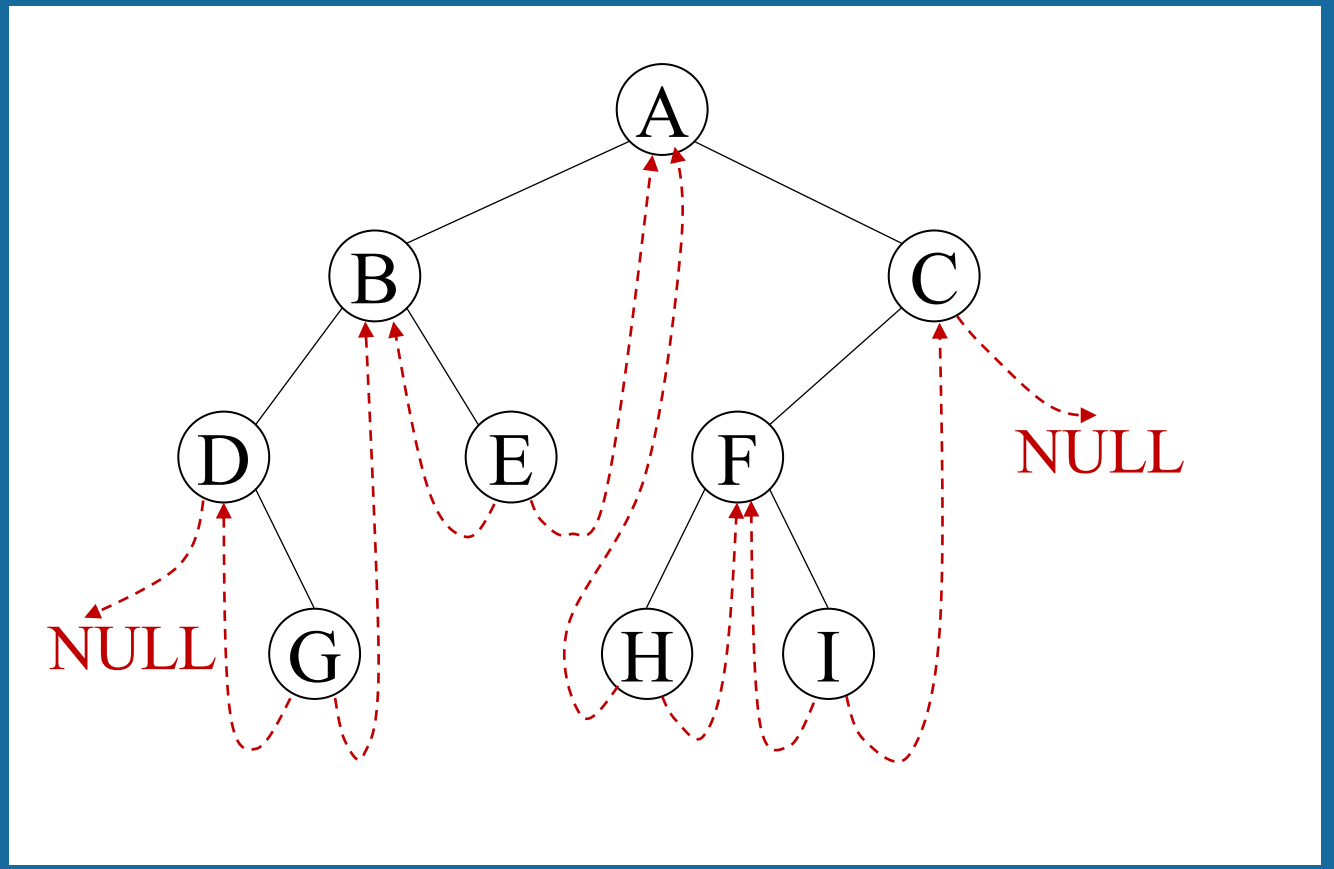
2+1+2+X=2*3+1*2+2*1+X*0+1，

X=6

7. 中序线索化

二叉树如下图所示，画出中序线索化后的穿线二叉树。





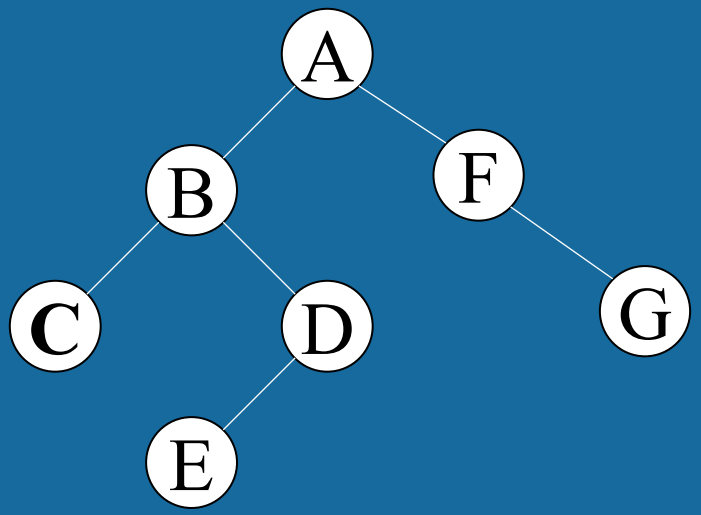
8. 已知前/中序遍历次序构造二叉树

设先序遍历某二叉树得到的结点访问序列为ABCD，中序遍历该二叉树的序列为BADC，则后序遍历该二叉树的序列为 BDCA 。

某二叉树结点的先序和中序访问次序如下，试画出该二叉树。

先序：ABCDEFG

中序：CBEDAFG

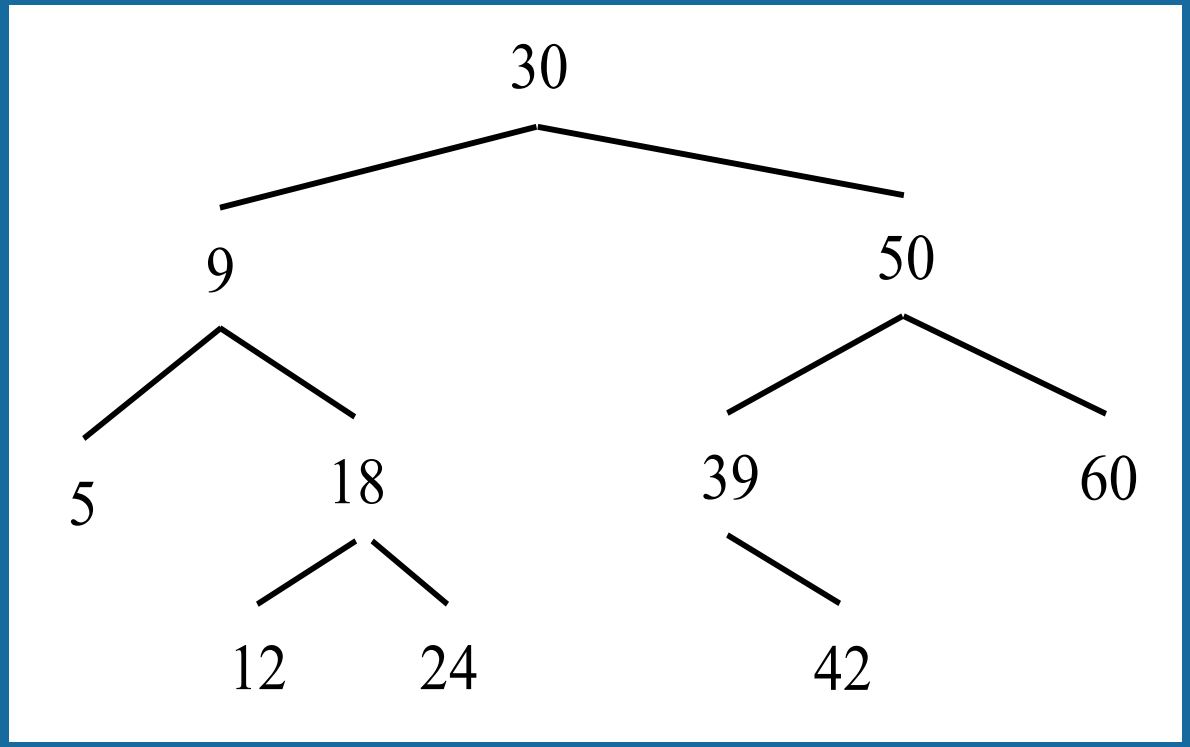


8. 已知前/中序遍历次序构造二叉树

续1完

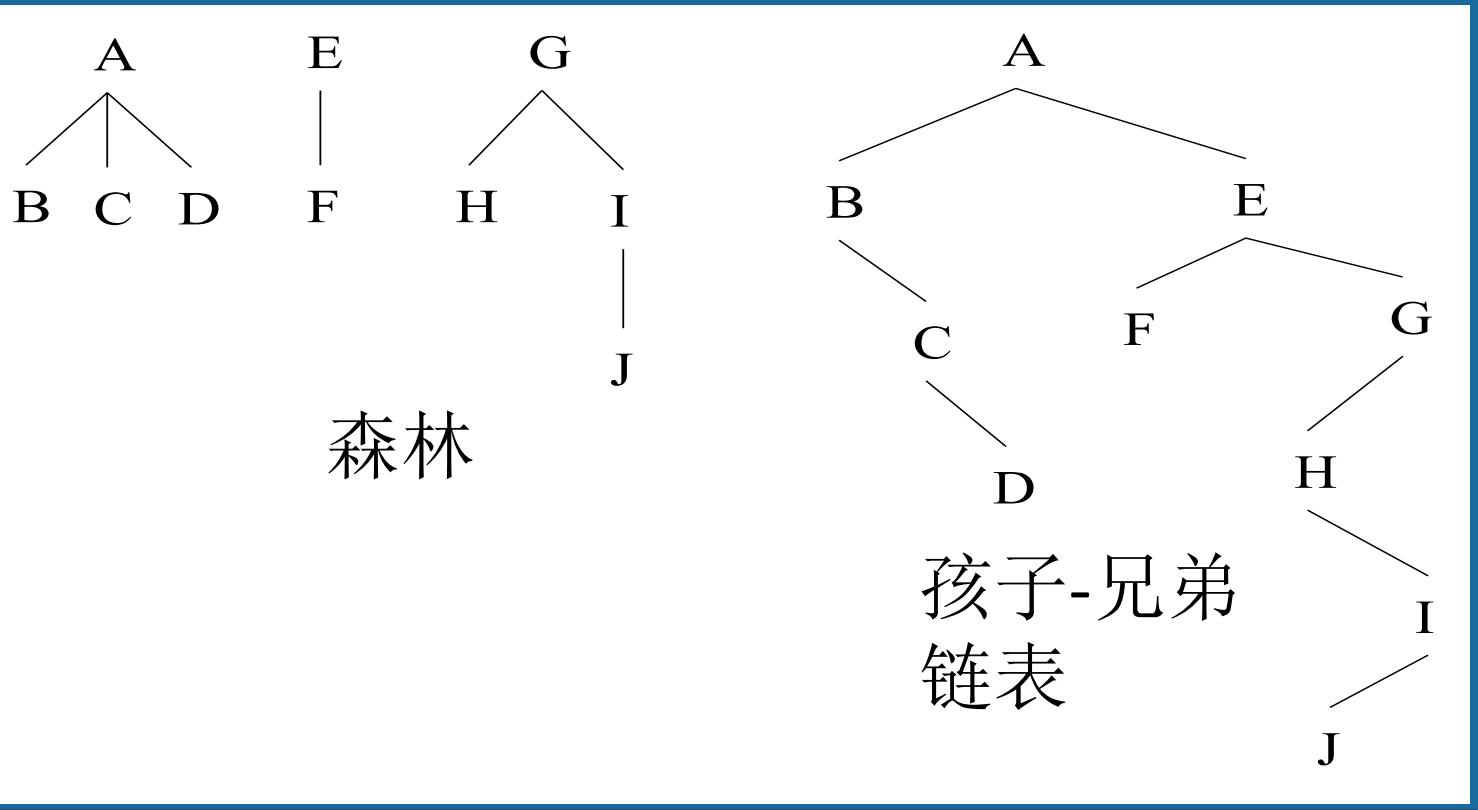
已知10个关键字组成的某二叉排序树的先序遍历关关键字次序为30, 9, 5, 18, 12, 24, 50, 39, 42, 60。画出该二叉排序树。

答：中序遍历二叉排序树，关键字访问次序为升序（故其中序遍历结点访问次序已知），得二叉树为：



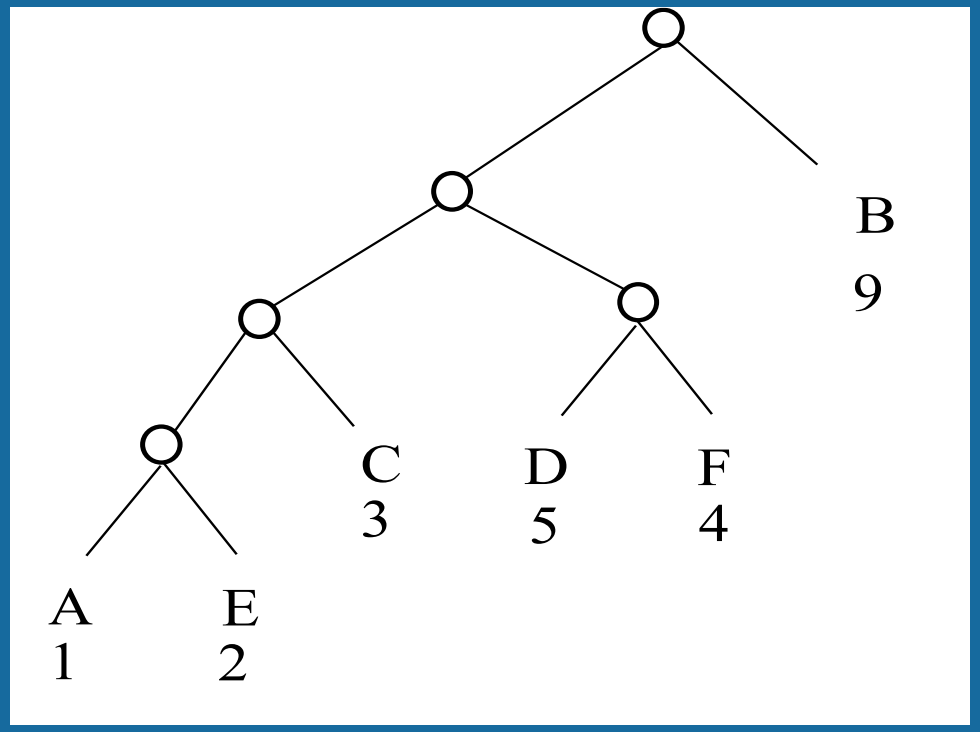
9. 树与森林的转换-孩子兄弟链表

由三棵树组成的森林由广义表(A(B, C, D), E(F), G(H, I(J)))给出，请画出该森林；若用孩子兄弟链表将该森林表示为二叉树形式，画出该二叉树。



10. 哈夫曼二叉树与哈夫曼编码, 平均码长的计算

六个符号A，B，C，D，E，F出现的概率分别是1/24, 9/24, 3/24, 5/24, 2/24, 4/24，建立一棵哈夫曼(Huffman)二叉树并计算这六个字符的平均编码长度。



WPL=((1+2)*4+(3+4+5)*3+1*9)/24

=(3*4+12*3+9)/24

=(12+36+9)/24

=57/24

=19/8

11. 图的存储结构(画图)与DFS、BFS遍历

用邻接表存储n个顶点e条边的无向图，其头（顶点）结点和表（边）结点的总数是 ___A___。

A. n+2e　　B. e　　C. 2e　　D. n+e

某DAG的邻接表存储结构如下图所示，基于该存储结构：

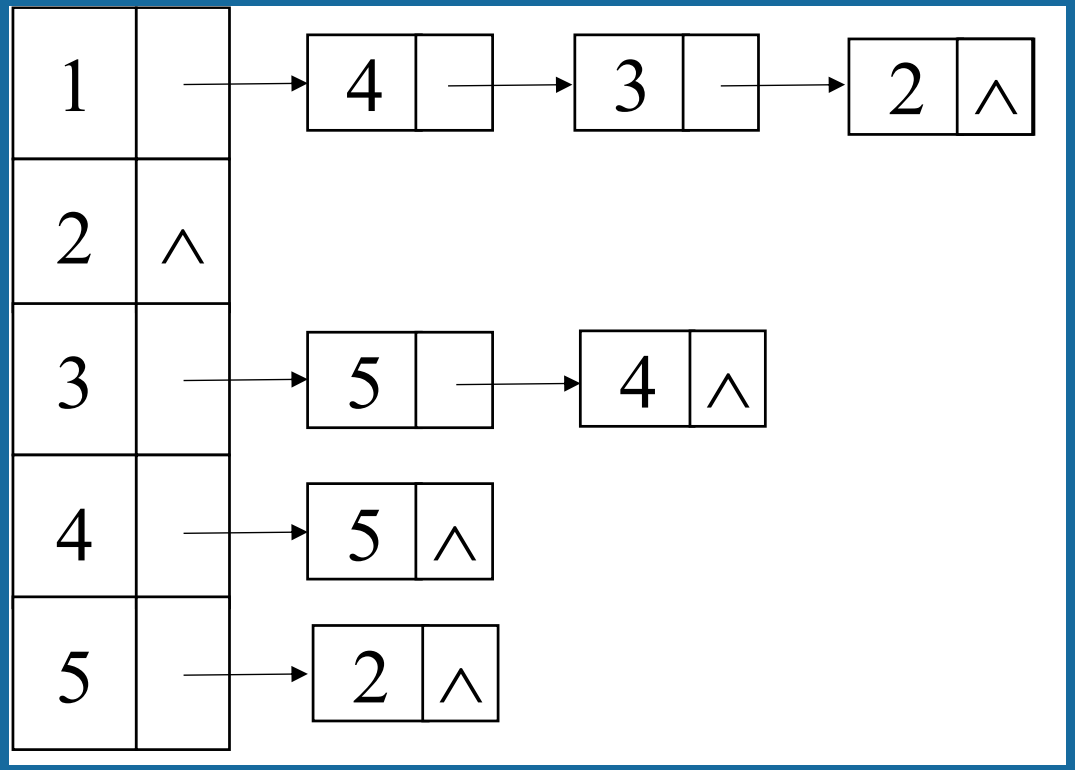
(1) 画出该DAG；

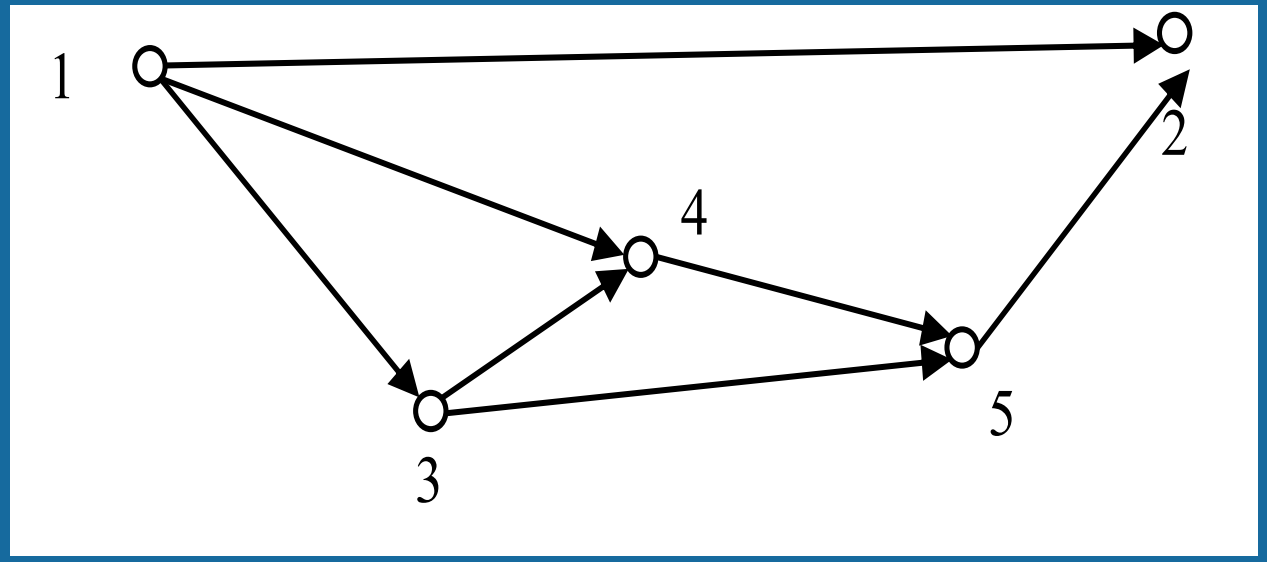
(2) 写出从1号顶点出发的深度优先遍历顶点访问次序；

(3) 写出从1号顶点出发的宽度优先遍历顶点访问次序；

11. 图的存储结构(画图)与DFS、 BFS遍历

续1完





DFS: 1, 4, 5, 2, 3

BFS: 1, 4, 3, 2, 5

12. 中序穿线二叉树

已知中序穿线二叉树结点及结点指针类型定义如下：

```
typedef struct bt_node {
    char data; int ltag, rtag; struct bt_node *lchild, *rchild;
} BTNode, *BT;
```

若ltag==0为真，则lchild指向中序遍历次序的前驱结点，否则lchild指向左儿子；

若rtag==0为真，则rchild指向中序遍历次序的后继结点，否则rchild指向右儿子。

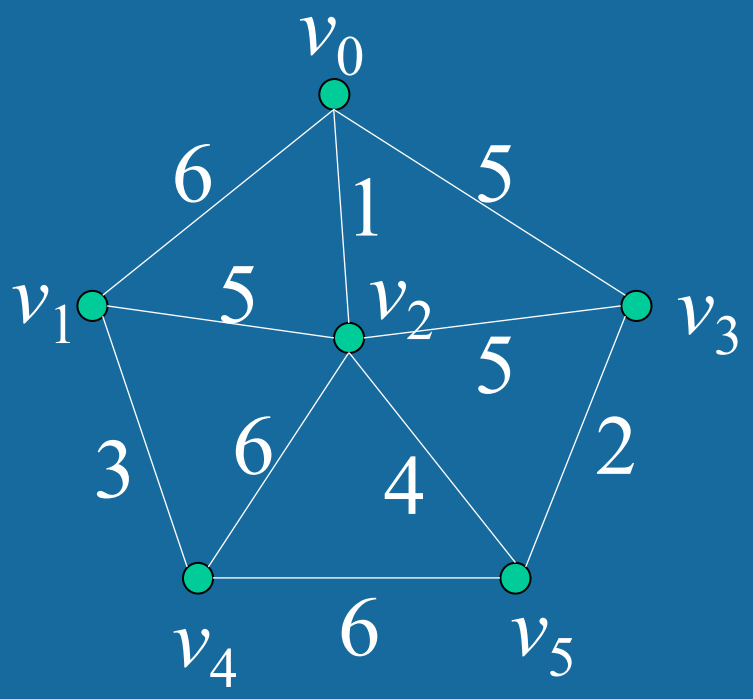
假设二叉树已完成中序线索化，以下算法函数利用线索指针实现非递归的中序遍历，请填空使算法完整。

12. 中序穿线二叉树

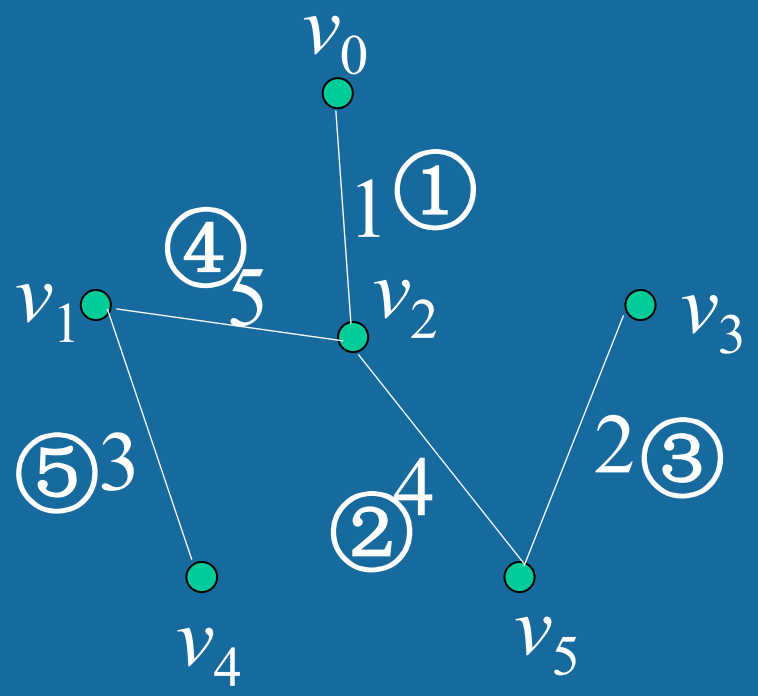
续1完

```
LPtr first(LPtr root)
{ if(!root) return NULL;
   LPtr p=root; while(p->ltag!=0)  p=p->lchild   ;
   return p;
}
void midtravel(LPtr bt)
{ LPtr p=first(bt);
  while(   p      )
{ visit(p);  /* 访问p所指结点 */
   if(  !p->rtag      ) p=p->rchild;      /* 线索后继 */
   else p=first(   p->rchild   );    /* 非线索后继 */
} }
```

13. 用普里姆或克鲁斯卡尔算法求最小生成树(画图)

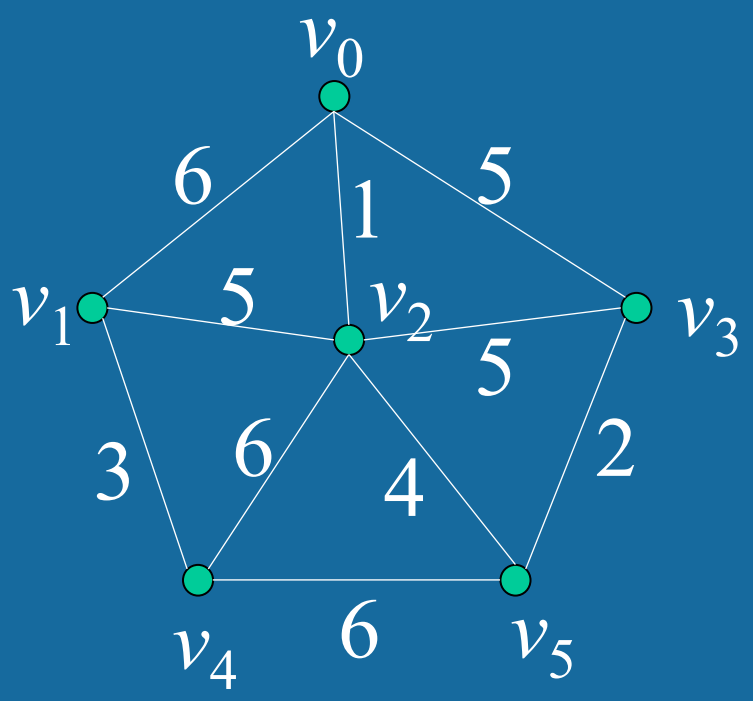


$U=\{v_0\}$开始

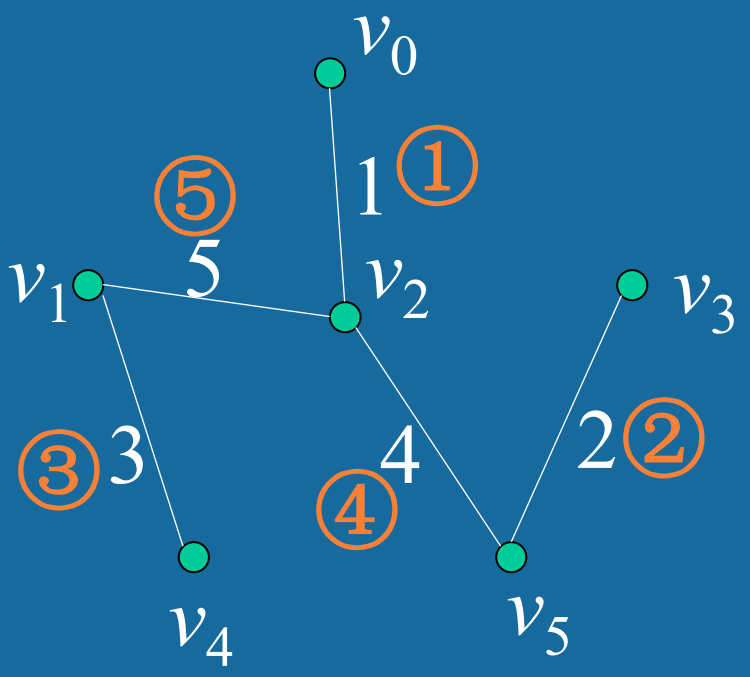


Prim算法（从v_0出发），圆圈数字表示边的生成次序

13. 用普里姆或克鲁斯卡尔算法求最小生成树(画图)

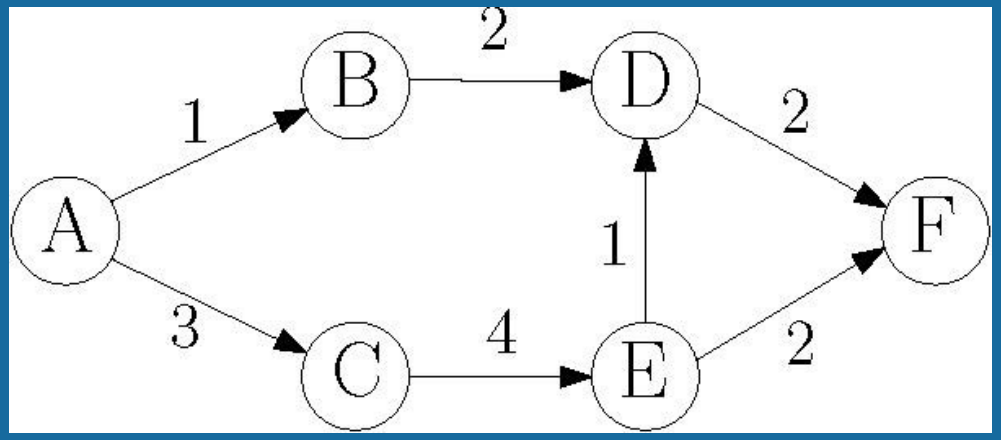


Kruskal算法



14. AOV网络与拓扑排序

某AOV网络如下图所示，若各顶点的邻接点次序为字母升序，写出采用队列和堆栈存储入度为零顶点时，拓扑排序算法得到的排序结果。



答:

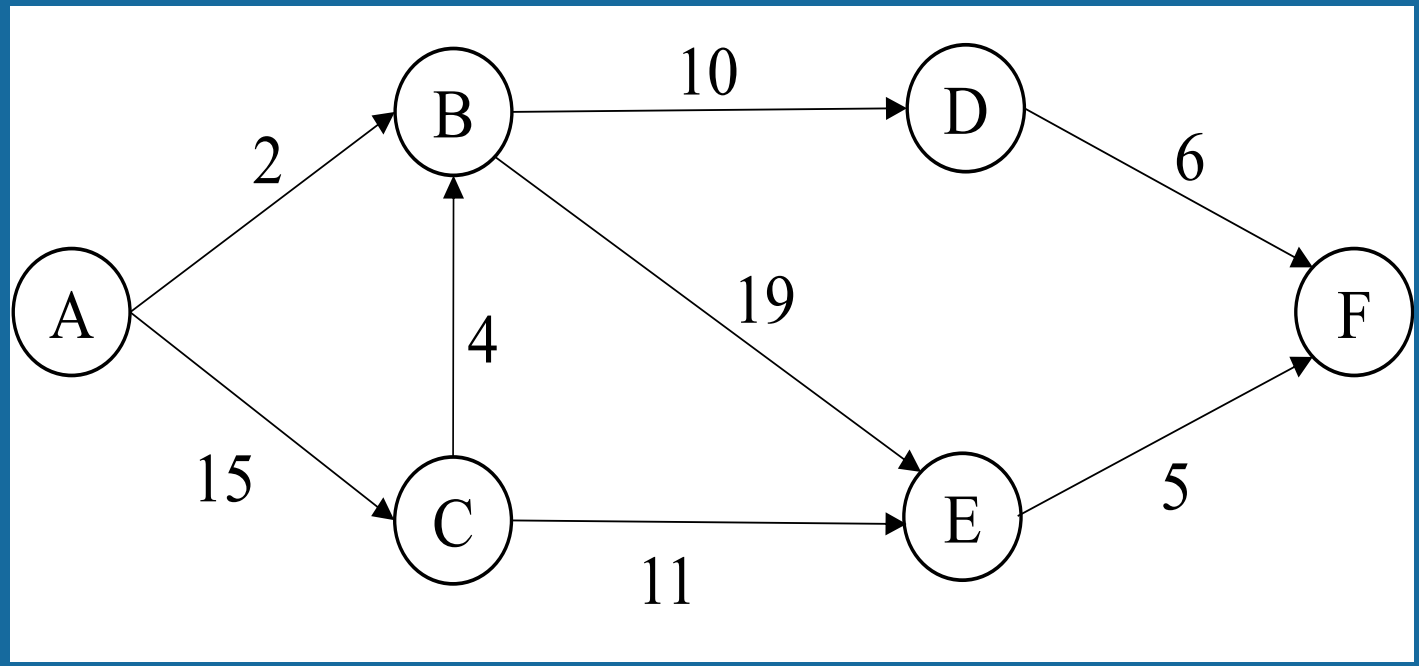
队列: ABCEDF

堆栈: ACEBDF

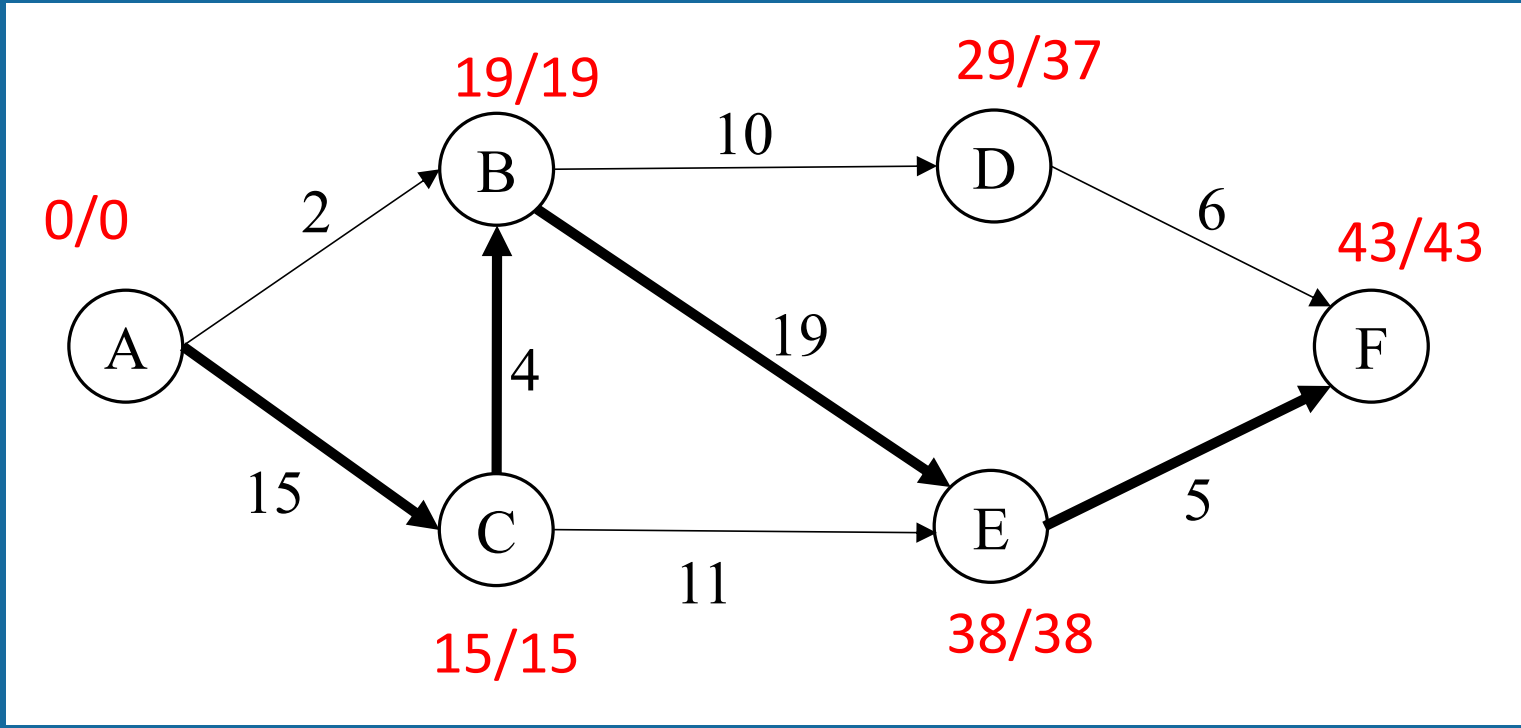
15. AOE网络(求顶点的最早开始/最晚开始时间，求关键活动及关键路径)

已知带权网络如下，边上的权重表示活动持续的时间，求解以下问题。(1) 写出各顶点的最早开始与最晚开始时间。

(2) 写出关键路径（用顶点拓扑有序序列表示）。



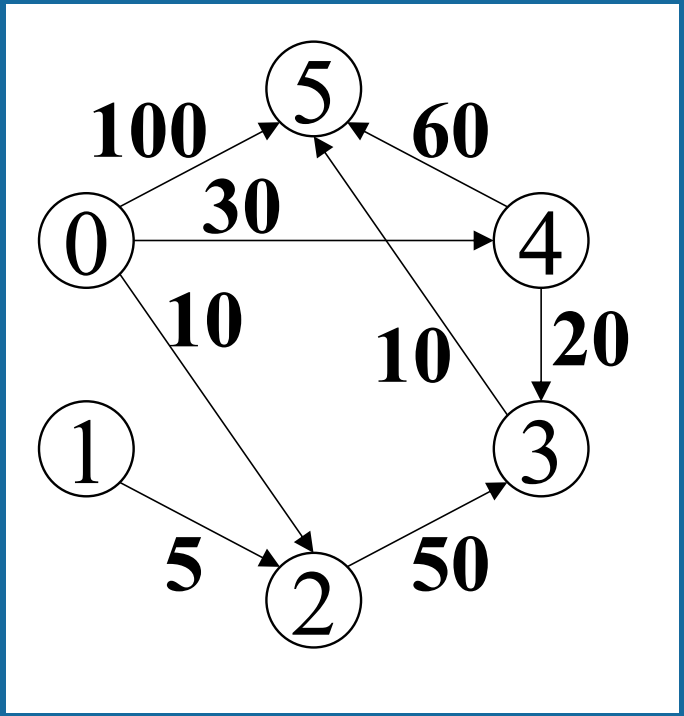
15. AOE网络(求顶点的最早开始/最晚开始时间，求关键活动及关键路径)



关键路径：ACBEF

16. 最短路径的计算

某有向网如右图所示。试写出0号顶点至1~5号顶点的最短路径长度及对应最短路径上的顶点序列。



答:

0-1： ∞(无路)

0-2： 10(0-2)

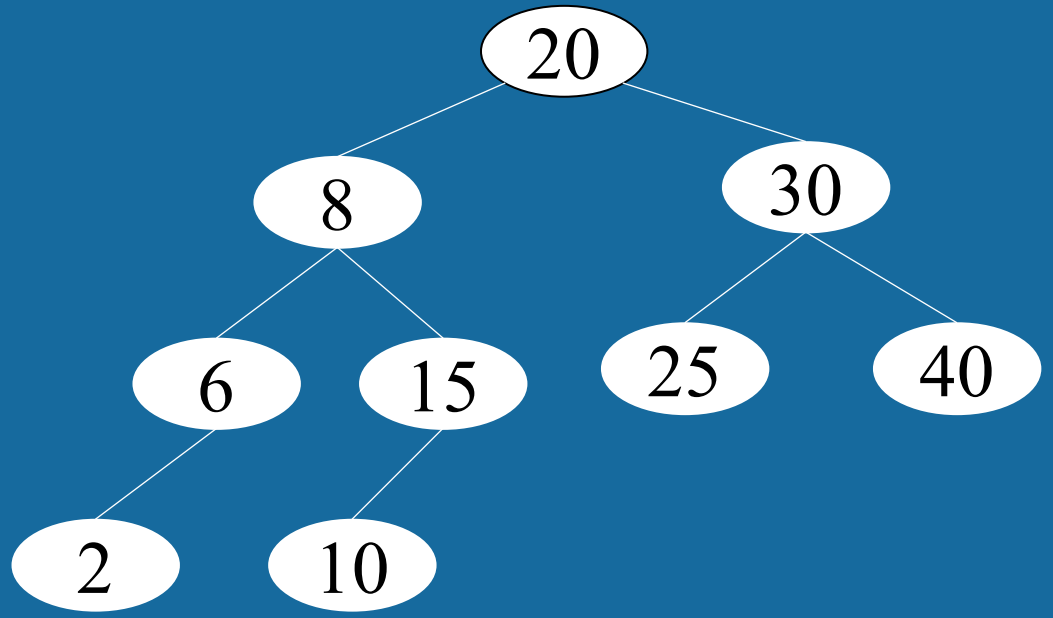
0-3： 50(0-4-3)

0-4： 30(0-4)

0-5： 60(0-4-3-5)

17. 二叉排序树插入与删除结点; 等概率成功查找时的ASL计算

9个关键字输入次序为20, 8, 30, 15, 6, 40, 10, 25, 2。按此输入次序构造二叉排序树；写出中序遍历次序；若每个元素等概率查找，试计算成功查找时的ASL。



中序遍历次序:

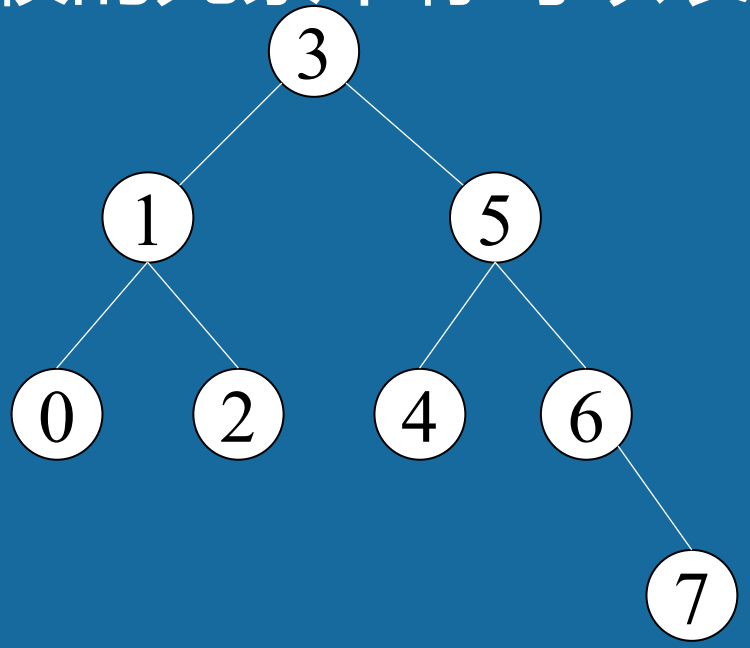
2,6,8,10,15,20,25,30,40

$$ASL_{成功} = \frac{1}{9}(1\cdot 1 + 2\cdot 2 + 3\cdot 4 + 4\cdot 2) = \frac{25}{9}$$

18. 对分(折半)查找的二叉判定树以及ASL计算

若8元升序数组每个元素等概率查找，分析对分查找的平均查找长度。

解：设8个元素按升序排列为$a_0, a_1, \ldots, a_7$, 则对分查找每次循环比较的元素下标可以表示为如下二叉判定树：



$$\mathrm{ASL}_{\text{成功}} = \frac{1}{8}\left(1 + 2\cdot 2 + 3\cdot 4 + 4\cdot 1\right) = 2.625$$

19. 分块顺序查找的最佳分块方法

900个元素的索引顺序表，分块内部和索引表均用顺序查找，则最佳分块长度为___30_____。

元素个数为n，则最佳分块长度= $\sqrt{n}$

20. 哈希表

1. 12个关键字为{19, 14, 23, 01, 68, 20, 84, 27, 55, 11, 10, 79}
设$H(K)=K \bmod 13$，地址空间范围0~15，用二次探测再散列解决冲突。画出哈希表；若各元素等概率查找，求成功查找时的平均查找长度。

解：$H_i=(H(K)\pm i^2) \bmod 13\ (i<7)$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
27	14	01	68	55	84	19	20		10	23	11		79		
3	1	2	1	2	3	1	1		3	1	1		5		

$$ASL_{成功}=(1\times6+2\times2+3\times3+5\times1)/12=2$$

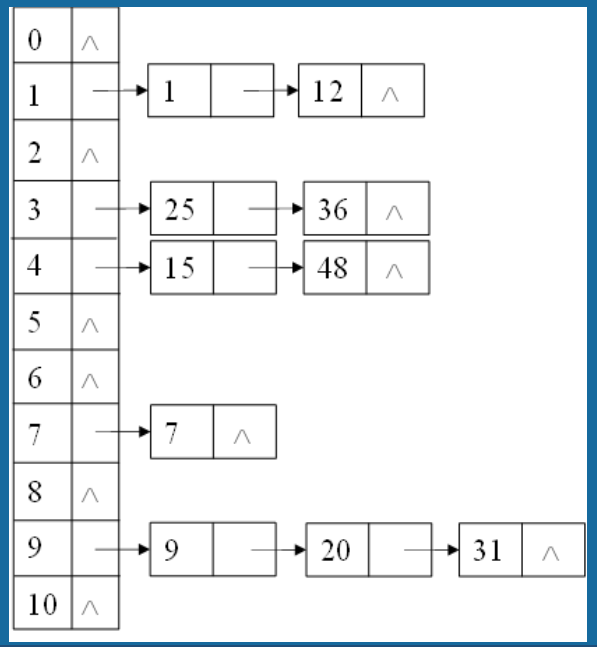
20. 哈希表

续1完

2. 使用散列函数h(x)=x mod 11(哈希表地址空间0..10)，把10个关键字9, 25, 15, 20, 7, 1, 36, 48, 12, 31存入哈希表中。

(1) 若使用链地址法解决冲突，试画出构造好的哈希表。

(2) 计算各关键字等概率查找条件下查找成功时的平均查找长度。



$ASL_{成功}=0.1\times(1\times5+2\times4+3\times1)=1.6$

21. 希尔排序

例 设递减增量序列为5, 3, 1, 对以下10元序列进行由小到大希尔排序，写出各趟增量排序结果。

49 38 65 97 76 13 27 $\overline{49}$ 55 04

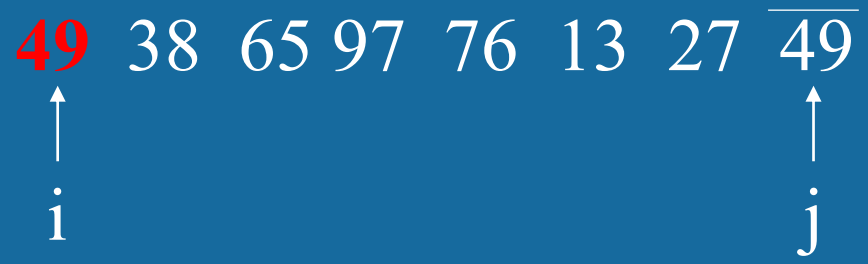
解：

元素	49	38	65	97	76	13	27	$\overline{49}$	55	04	第1趟
分组	**0**	**1**	**2**	**3**	**4**	**0**	**1**	**2**	**3**	**4**	增量=5
结果	13	27	$\overline{49}$	55	04	49	38	65	97	76	第2趟
分组	**0**	**1**	**2**	**0**	**1**	**2**	**0**	**1**	**2**	**0**	增量=3
结果	13	04	$\overline{49}$	38	27	49	55	65	97	76	第3趟
结果	04	13	27	38	$\overline{49}$	49	55	65	76	97	增量=1

21. 快速排序各趟排序的结果，支点元素的最终位置

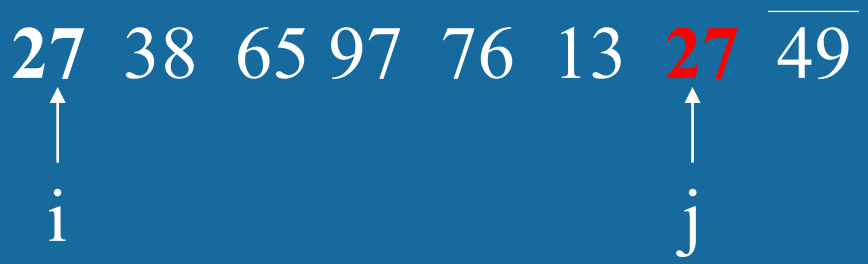
例 8个待排序关键字为49, 38, 65, 97, 76, 13, 27, $\overline{49}$, 排序下标范围0~7，以0号元素为支点进行非递减的快速排序，求一趟快排的结果以及支点元素的下标。

解：

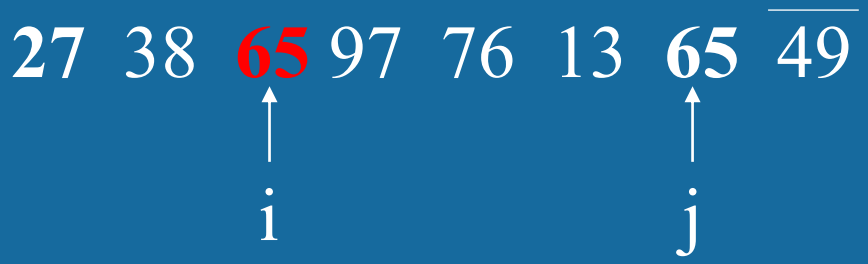


pivot=49

红色表示支点位置

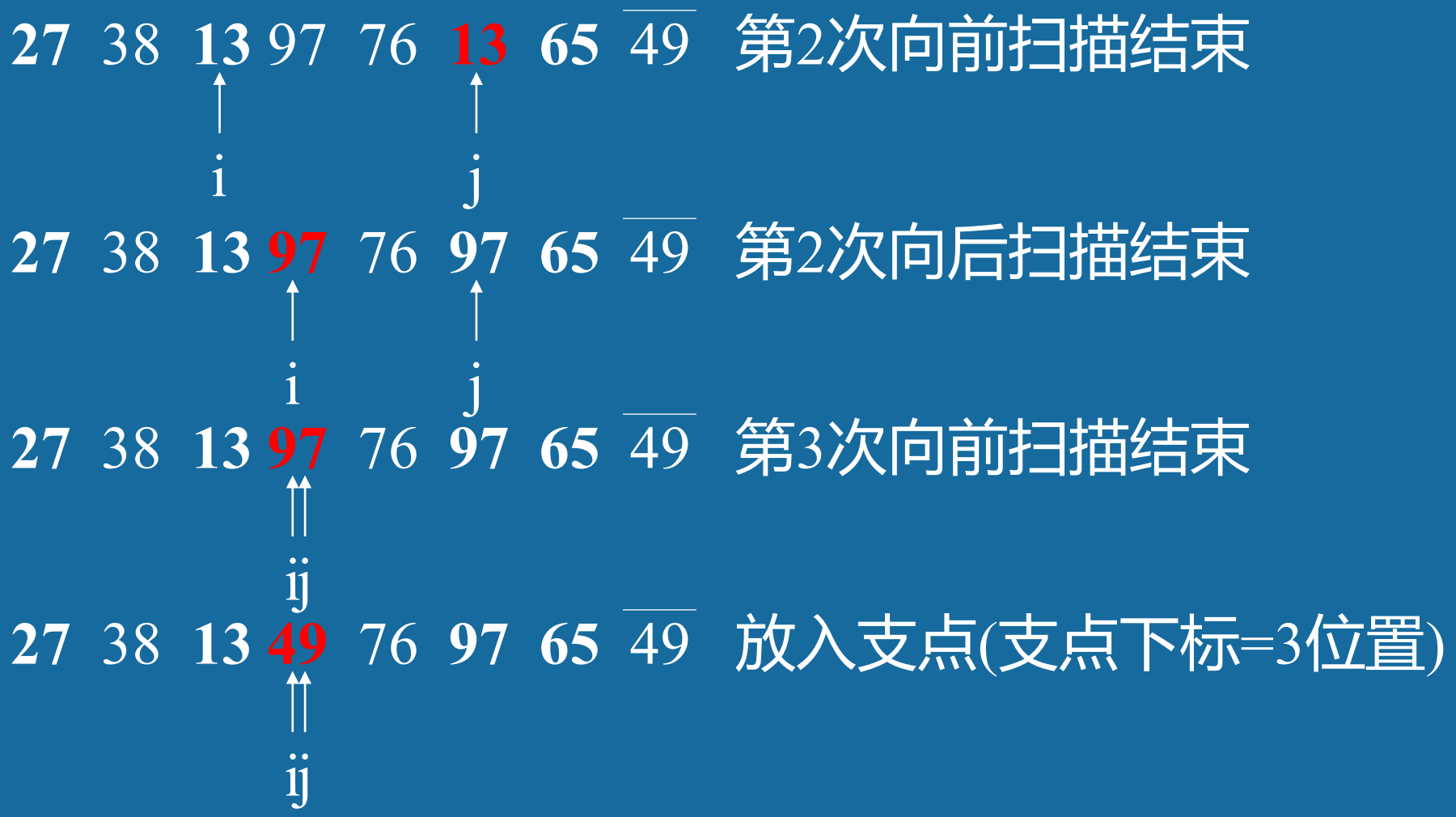


第1次向前扫描结束



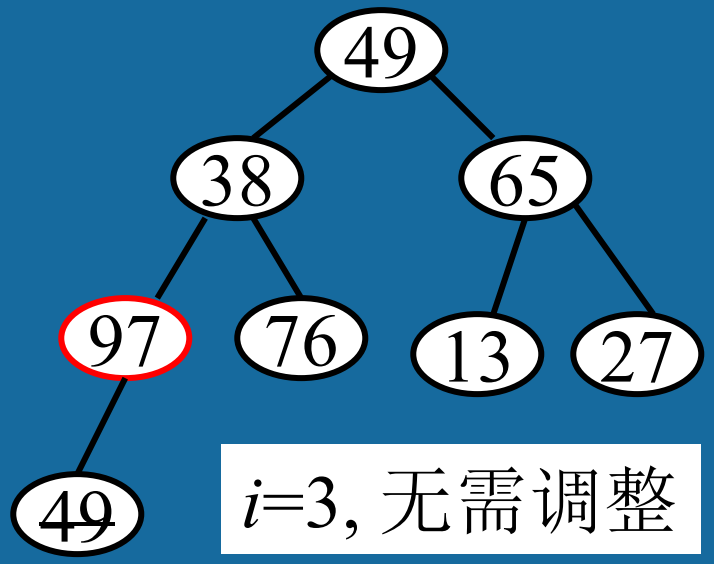
第1次向后扫描结束

22. 快速排序各趟排序的结果，支点元素的最终位置

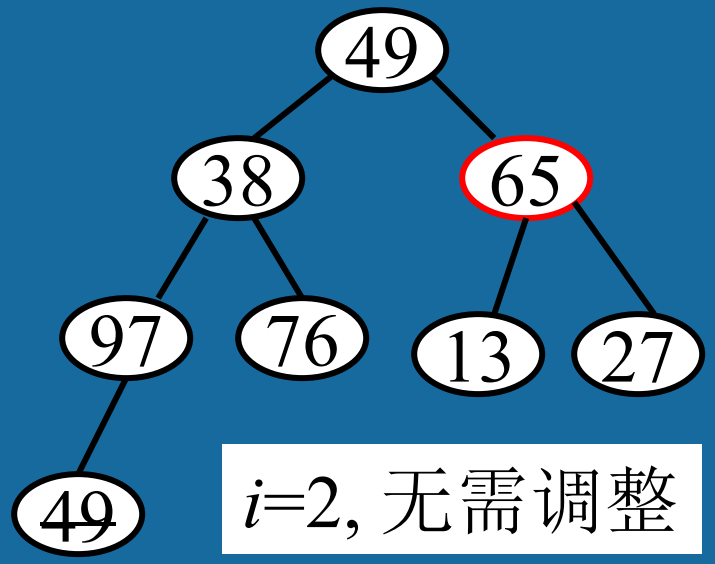


23. 堆排序初始建堆的结果，输出堆顶元素并进行调整后的结果

将序列{ 49, 38, 65, 97, 76, 13, 27, ~~49~~}构建为大根堆。

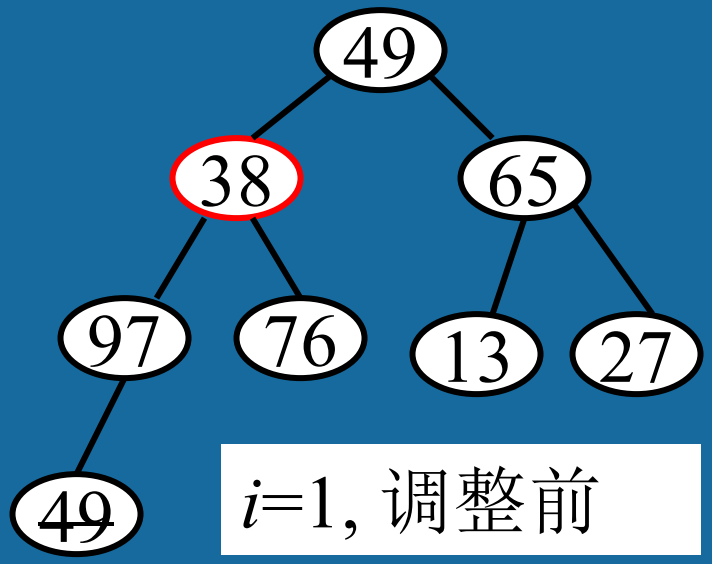


i=3, 无需调整

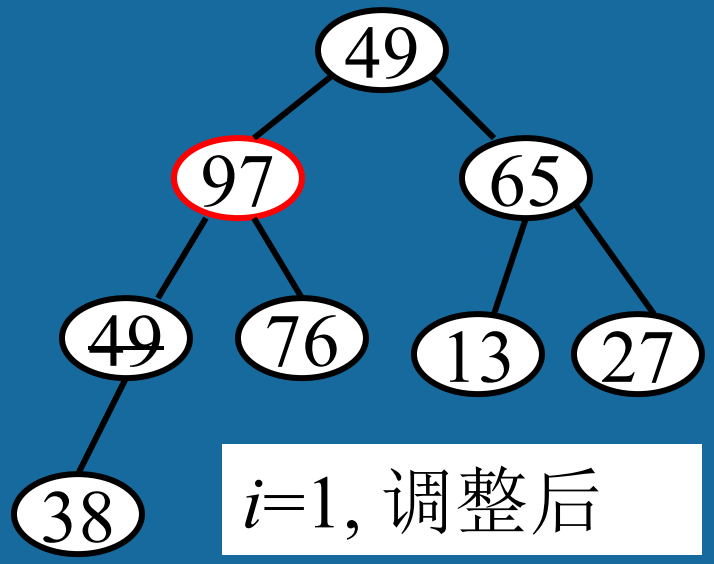


i=2, 无需调整

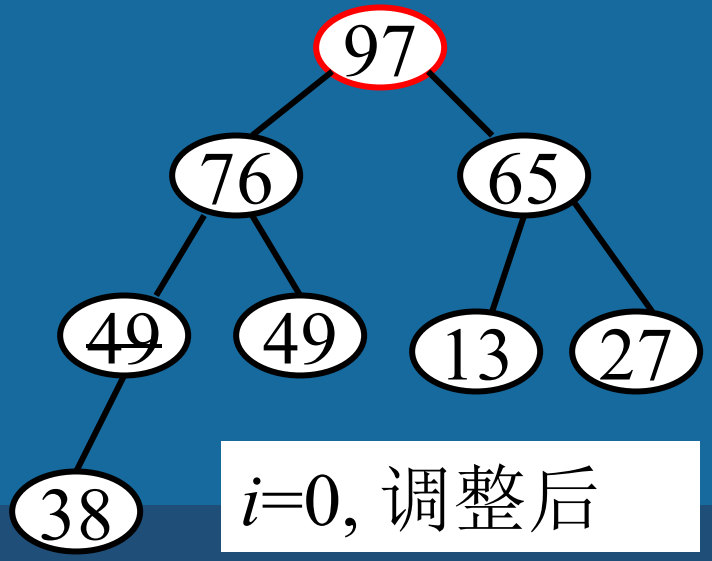
23. 堆排序初始建堆的结果，输出堆顶元素并进行调整后的结果



i=1, 调整前



i=1, 调整后



i=0, 调整后

故，初始建大根堆后的序列为

{ 97, 76, 65, ~~49~~, 49, 13, 27, 38 }

24. 归并排序各趟结果

例	{20}	{-8}	{9}	{26}	{7}	{-9}	{-26}	{34}	{15}	{-1}
第1趟归并后	{-8, 20}		{9, 26}		{-9, 7}		{-26, 34}		{-1, 15}	
第2趟归并后	{-8, 9, 20, 26}				{-26, -9, 7, 34}				{-1, 15}	
第3趟归并后	{-26, -9, -8, 7, 9, 20, 26, 34}								{-1, 15}	
第4趟归并后	{-26, -9, -8, -1, 7, 9, 15, 20, 26, 34}									

25. 写出KMP或改进KMP算法的next数组元素值

写出子串的next和nextval数组元素值。

s="aaab"

下标:	0	1	2	3
next	-1	0	1	2
nextval	-1	-1	-1	2

t="abcabaa"

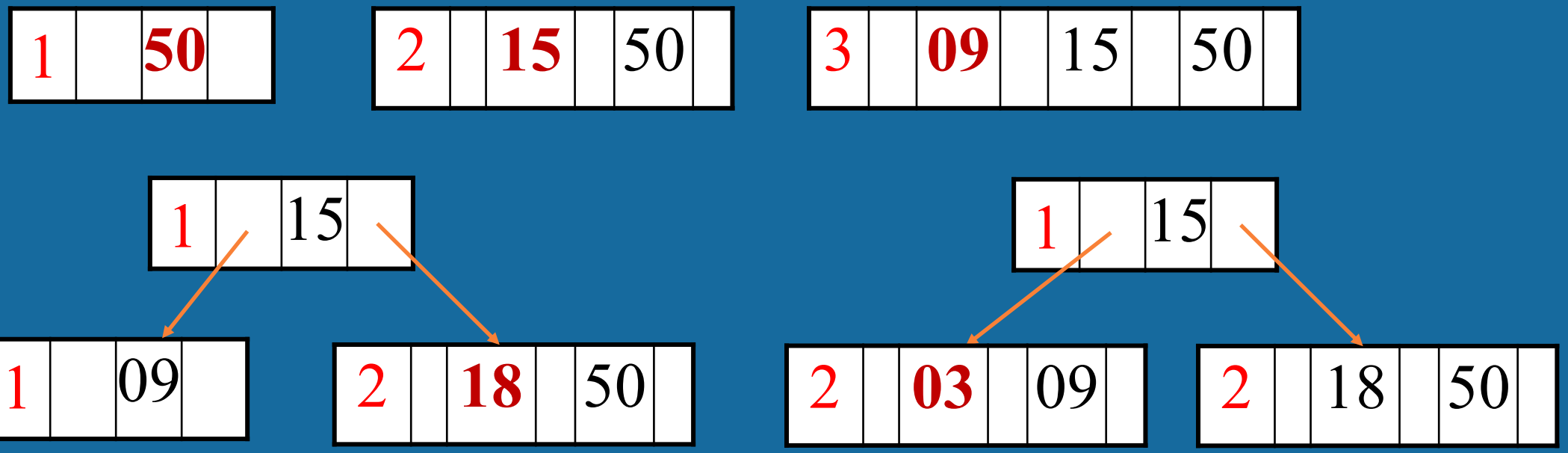
下标:	0	1	2	3	4	5	6
next	-1	0	0	0	1	2	1
nextval	-1	0	0	-1	0	2	0

26. B-、B+树插入/删除结点

从空树起连续插入以下20个关键字构建m=4的B-树。

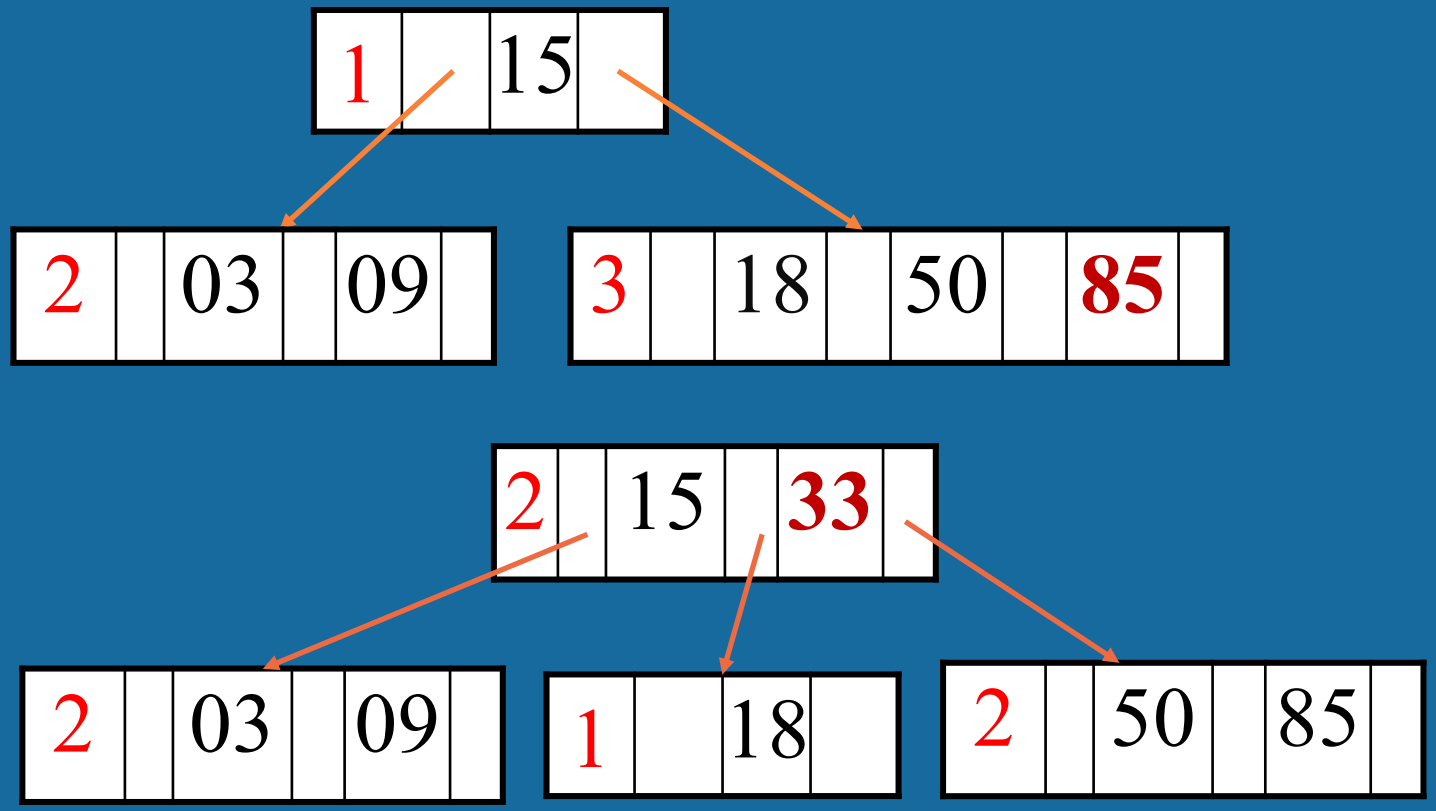
50, 15, 09, 18, 03, 85, 33, 72, 48, 22,

91, 88, 11, 99, 06, 56, 68, 77, 43, 36。



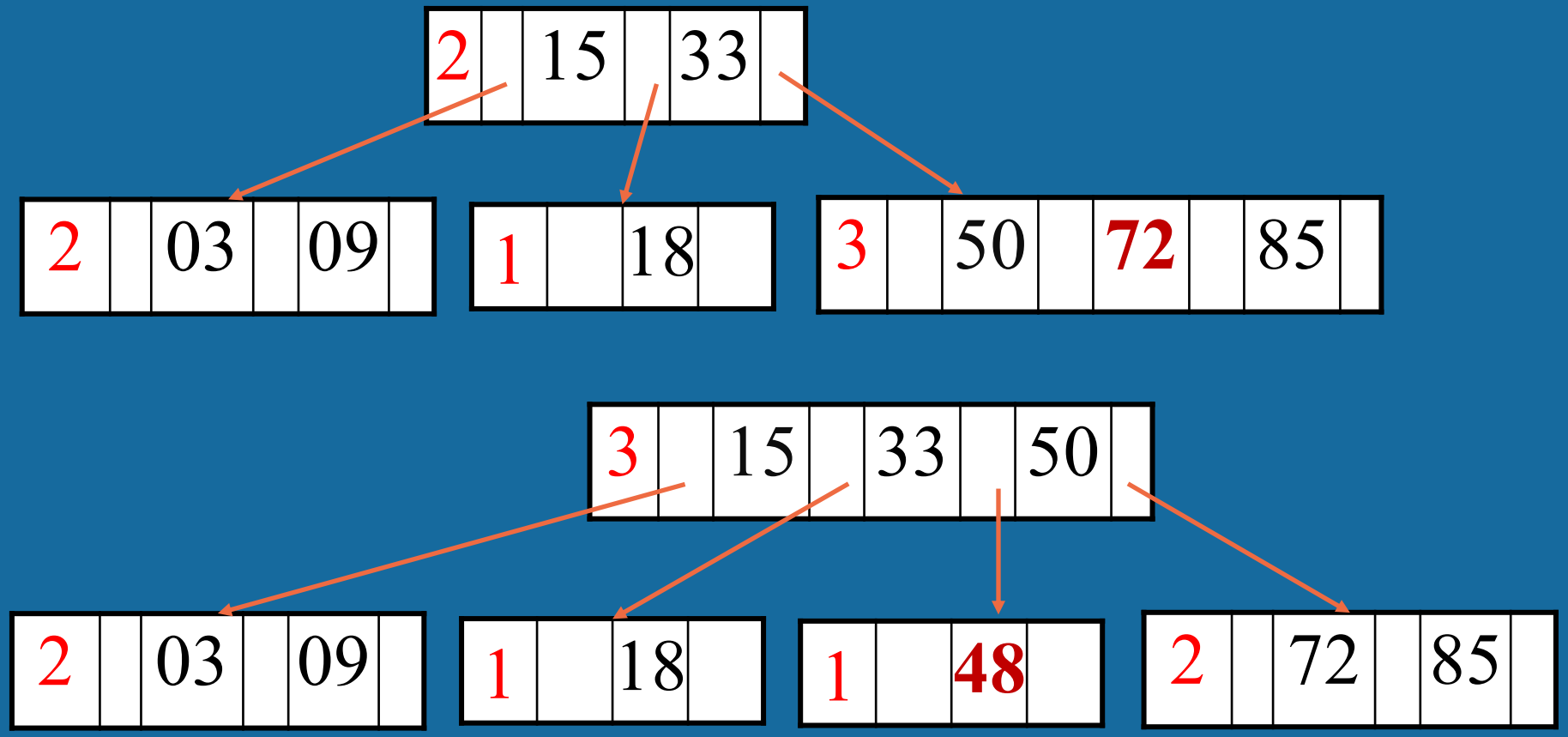
26. B-、B+树插入/删除结点

续2



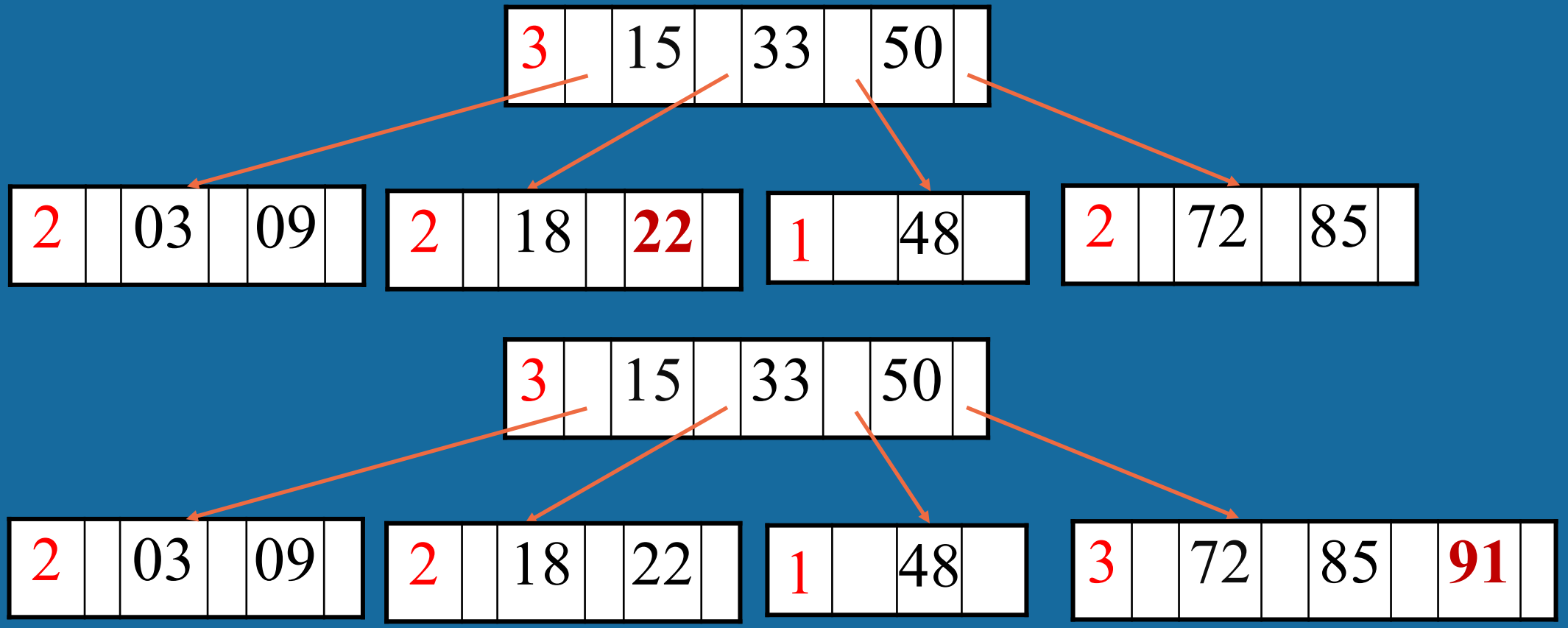
26. B-、B+树插入/删除结点

续3



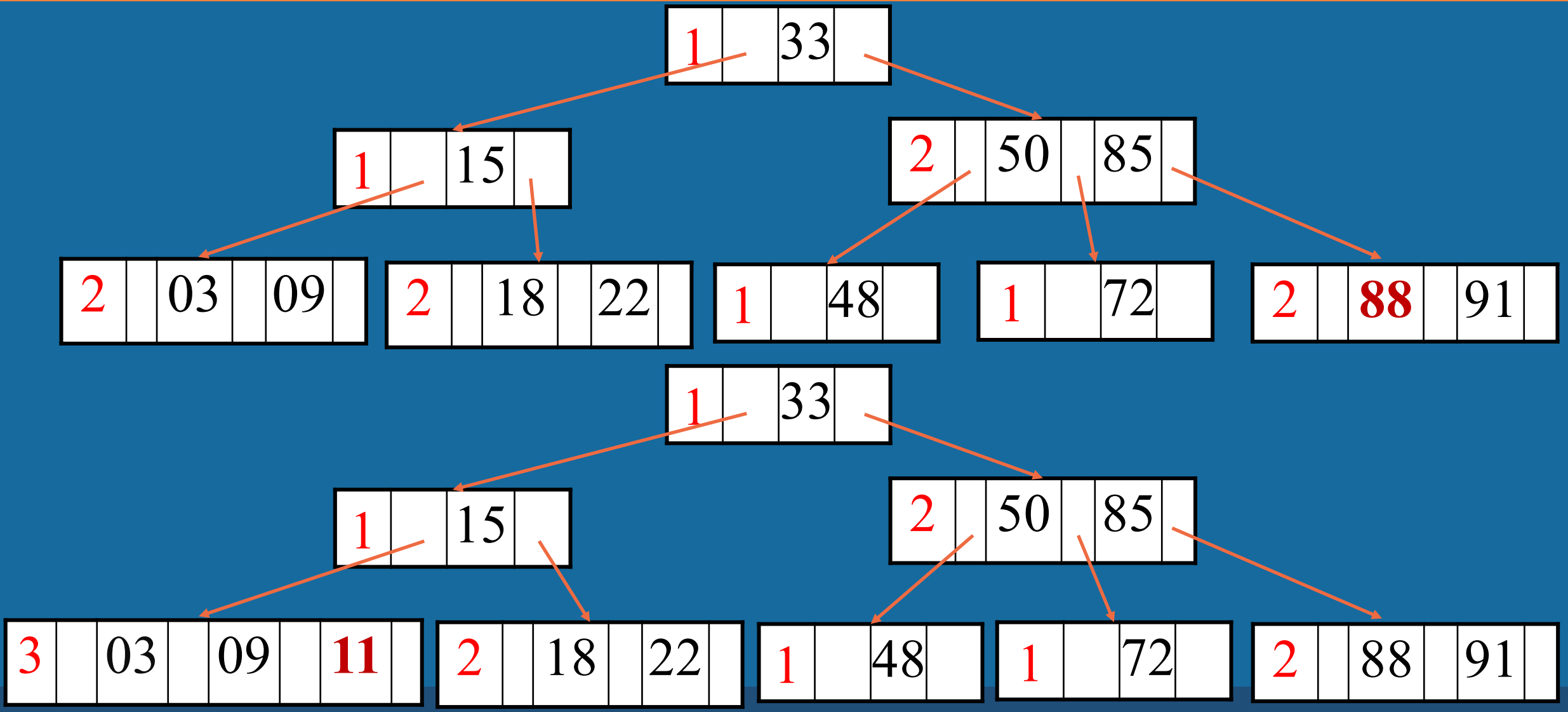
26. B-、B+树插入/删除结点

续4



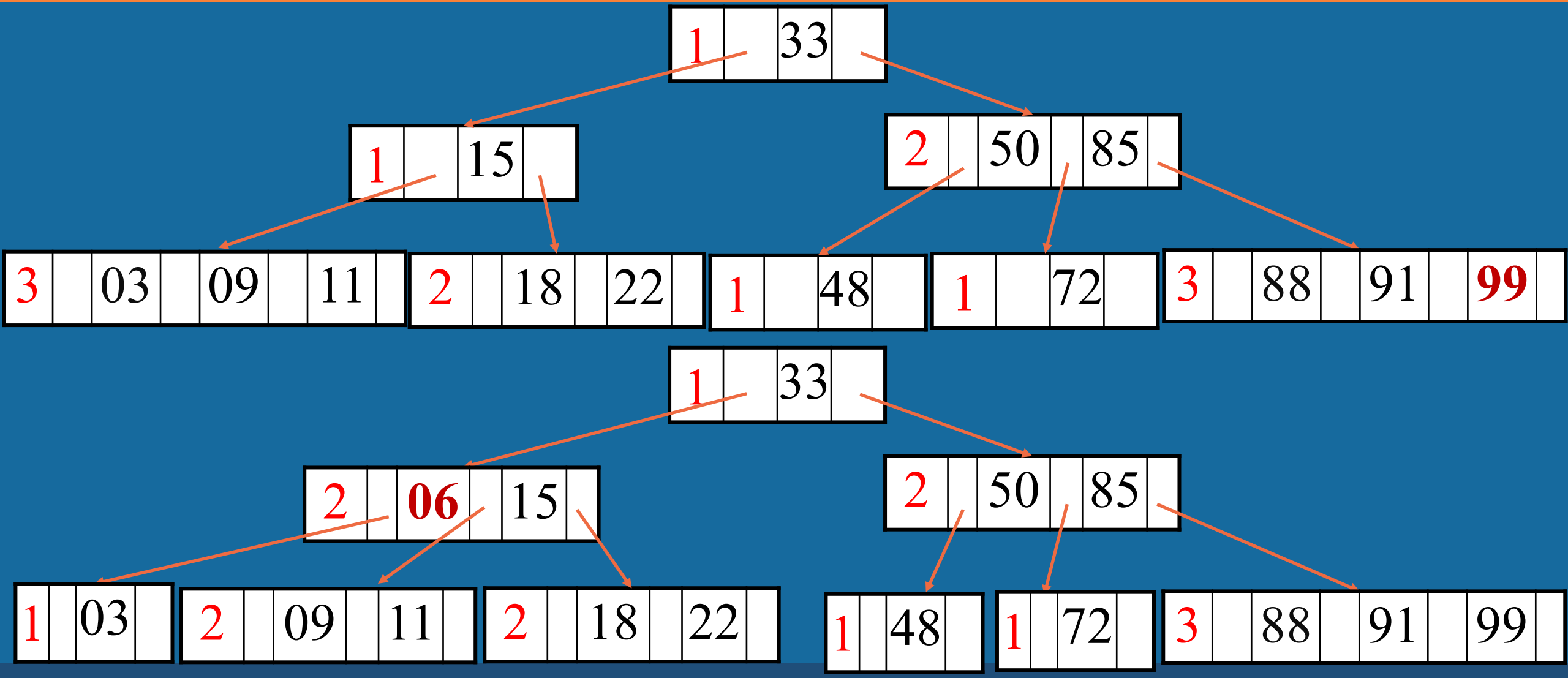
26. B-、B+树插入/删除结点

续5



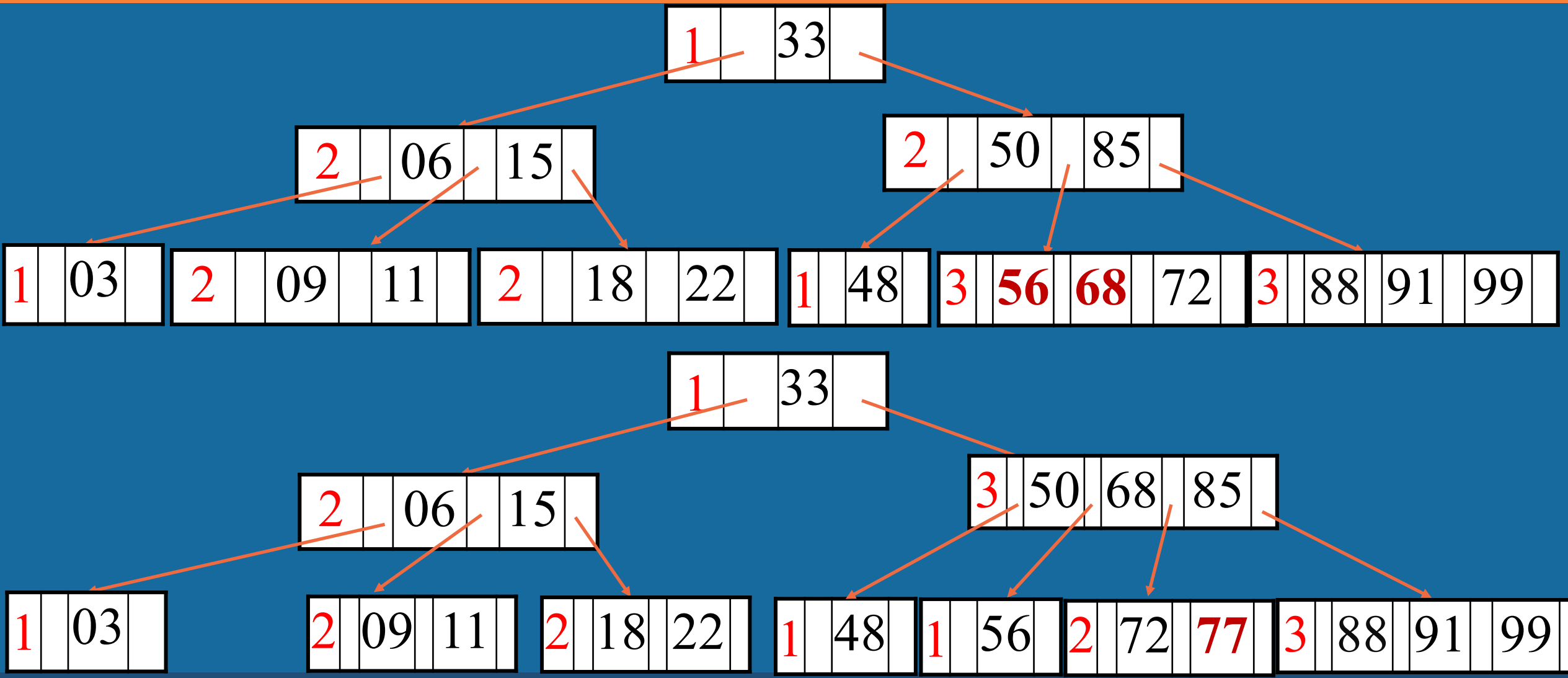
26. B-、B+树插入/删除结点

续6



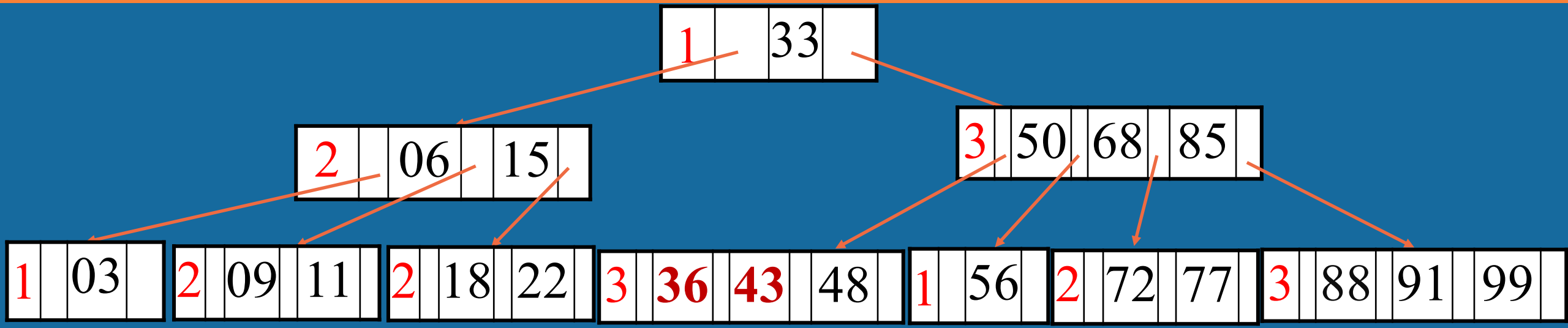
26. B-、B+树插入/删除结点

续7



26. B-、B+树插入/删除结点

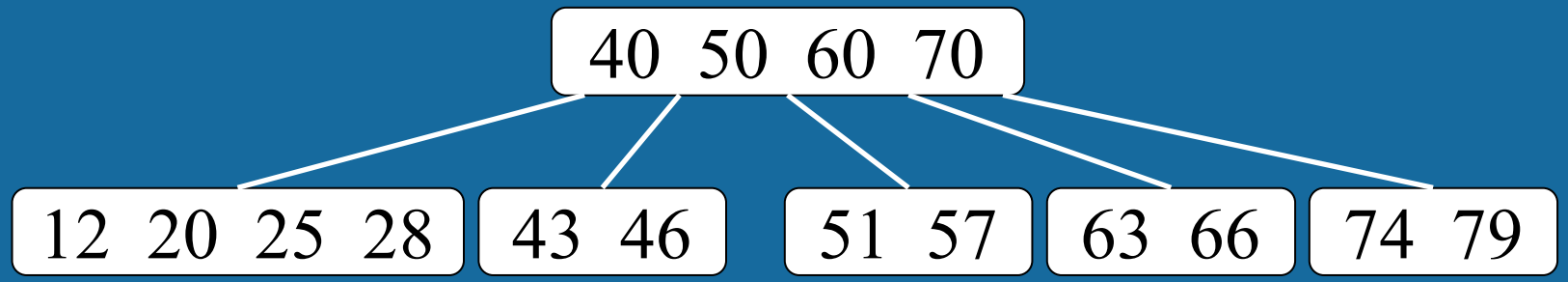
续8

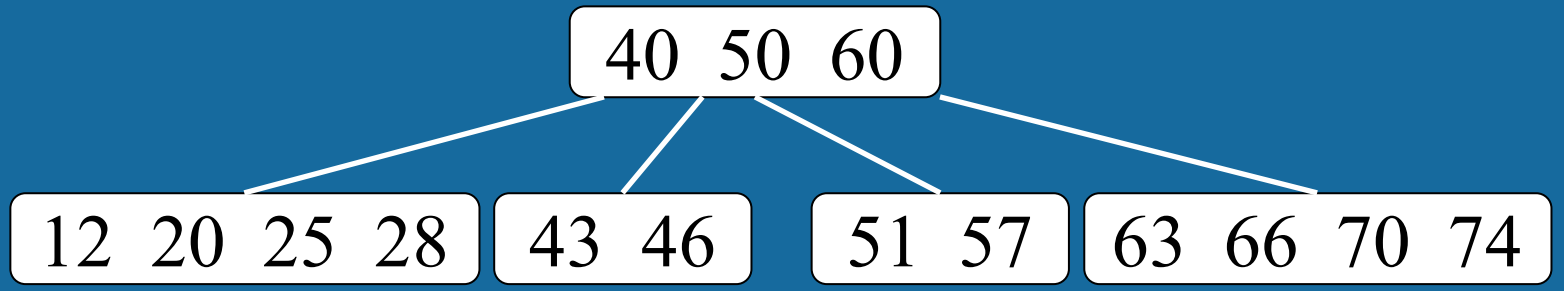


26. B-、B+树插入/删除结点

续9

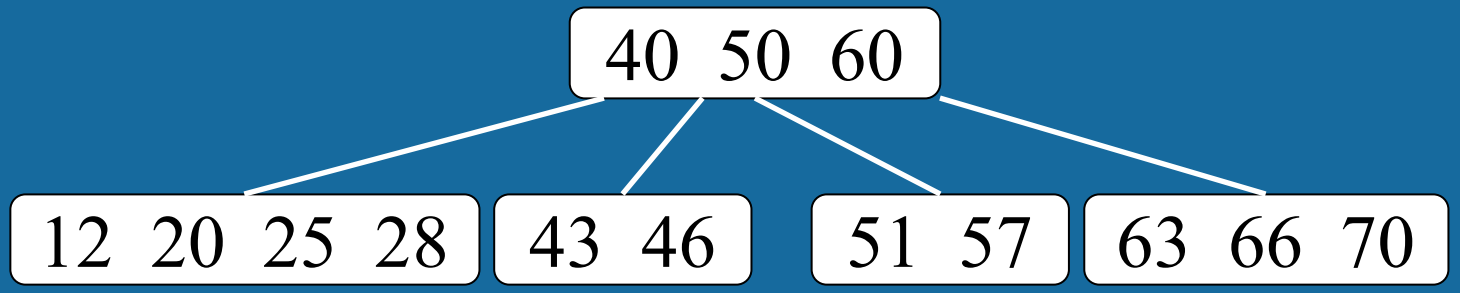
16个关键字组成的5阶B-树如下图所示，请按关键字递减的次序删除所有结点至空树，画出每删除1个关键字后得到B-树，直至空树。



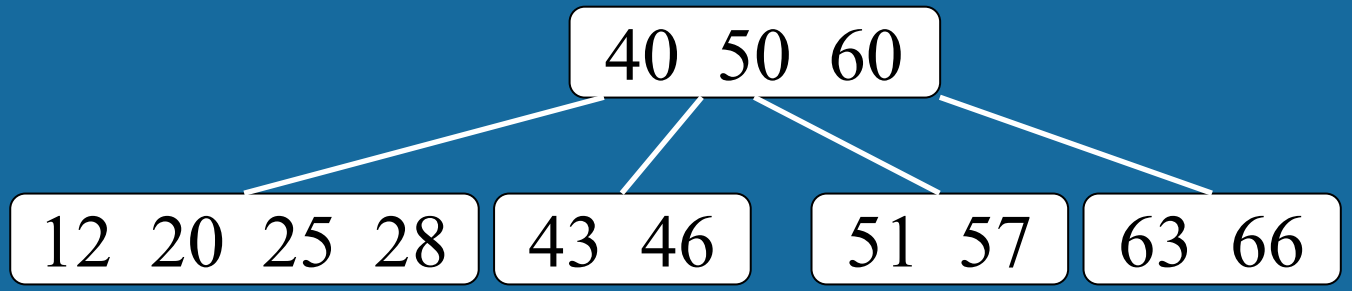


删除79后

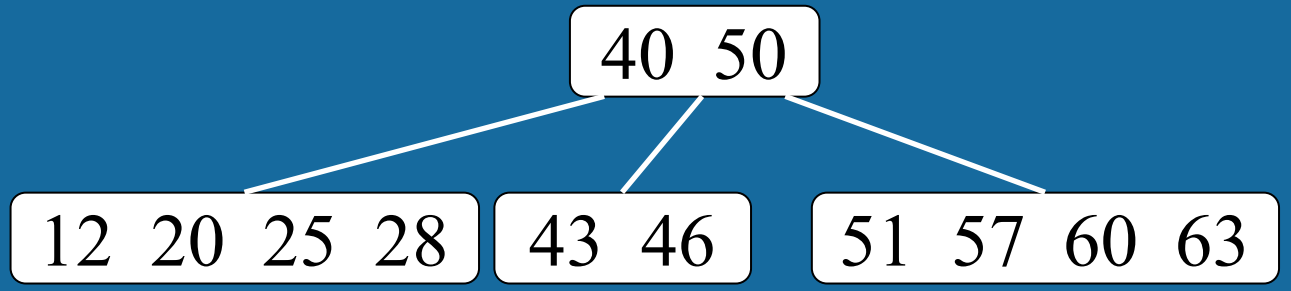
26. B-、B+树插入/删除结点



删除74后

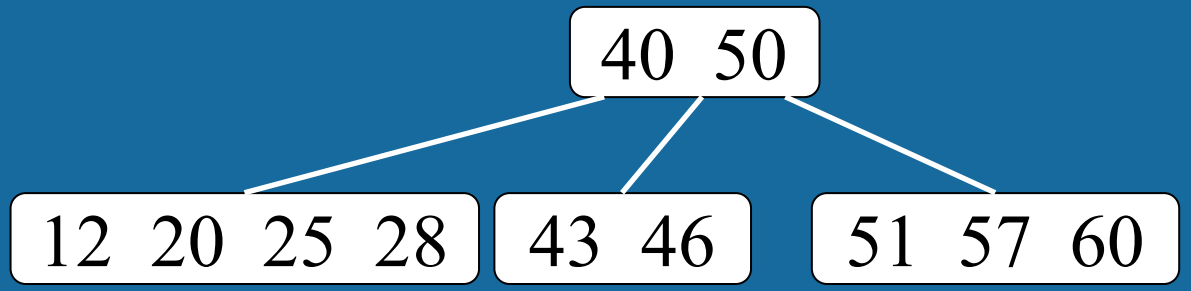


删除70后

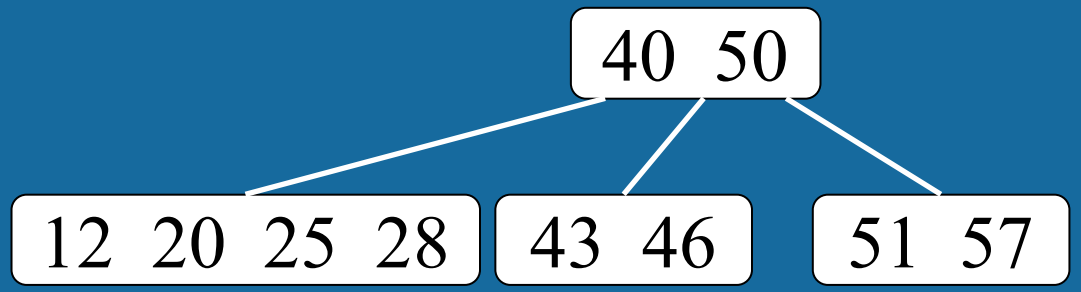


删除66后

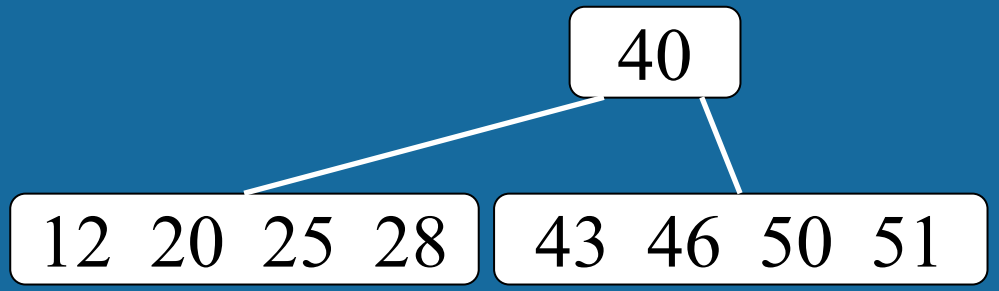
26. B-、B+树插入/删除结点



删除63后

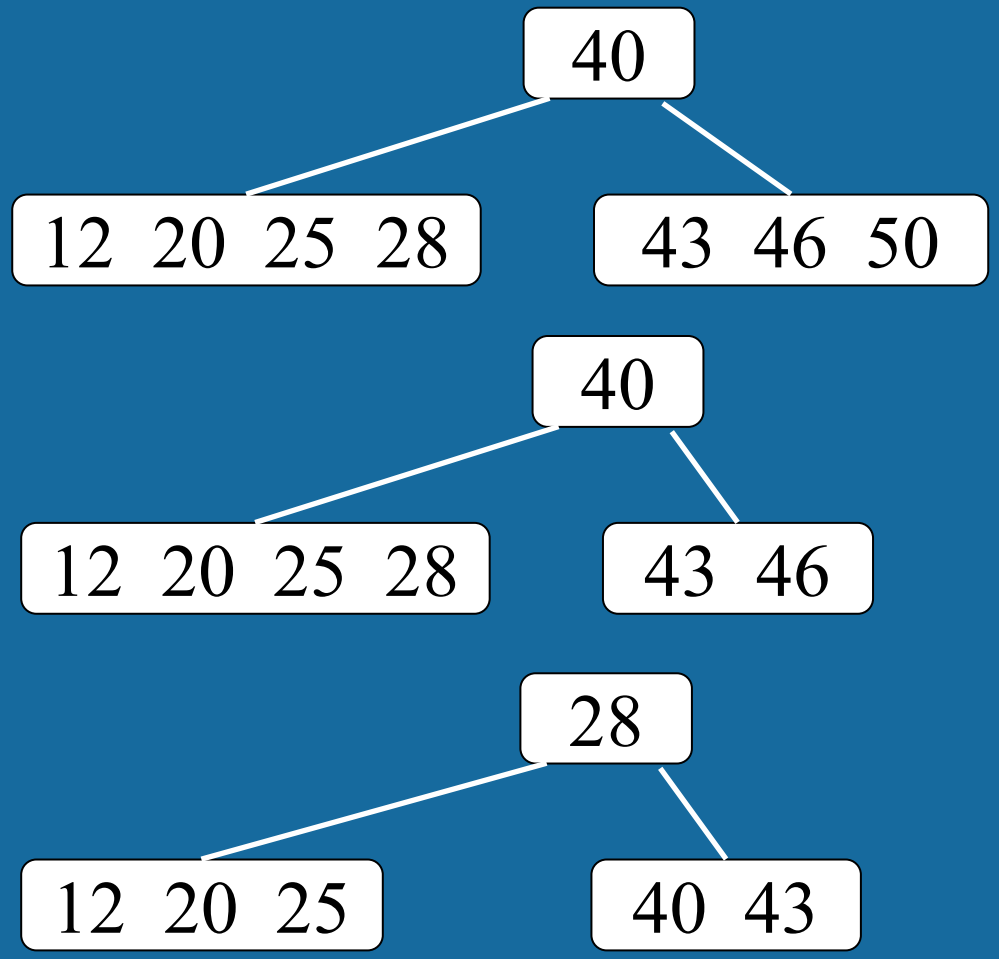


删除60后



删除57后

26. B-、B+树插入/删除结点



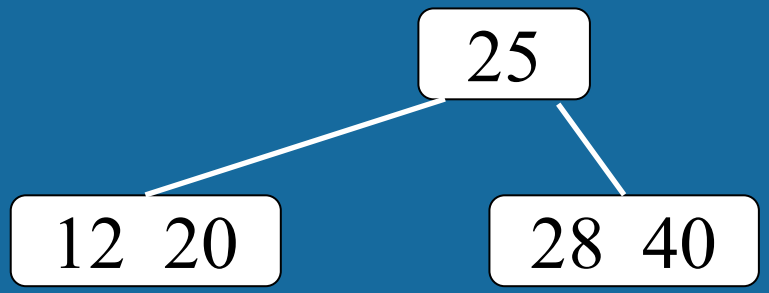
删除51后

删除50后

删除46后

26. B-、B+树插入/删除结点

续13完



删除43后



删除40后

27. 平衡二叉排序树插入结点

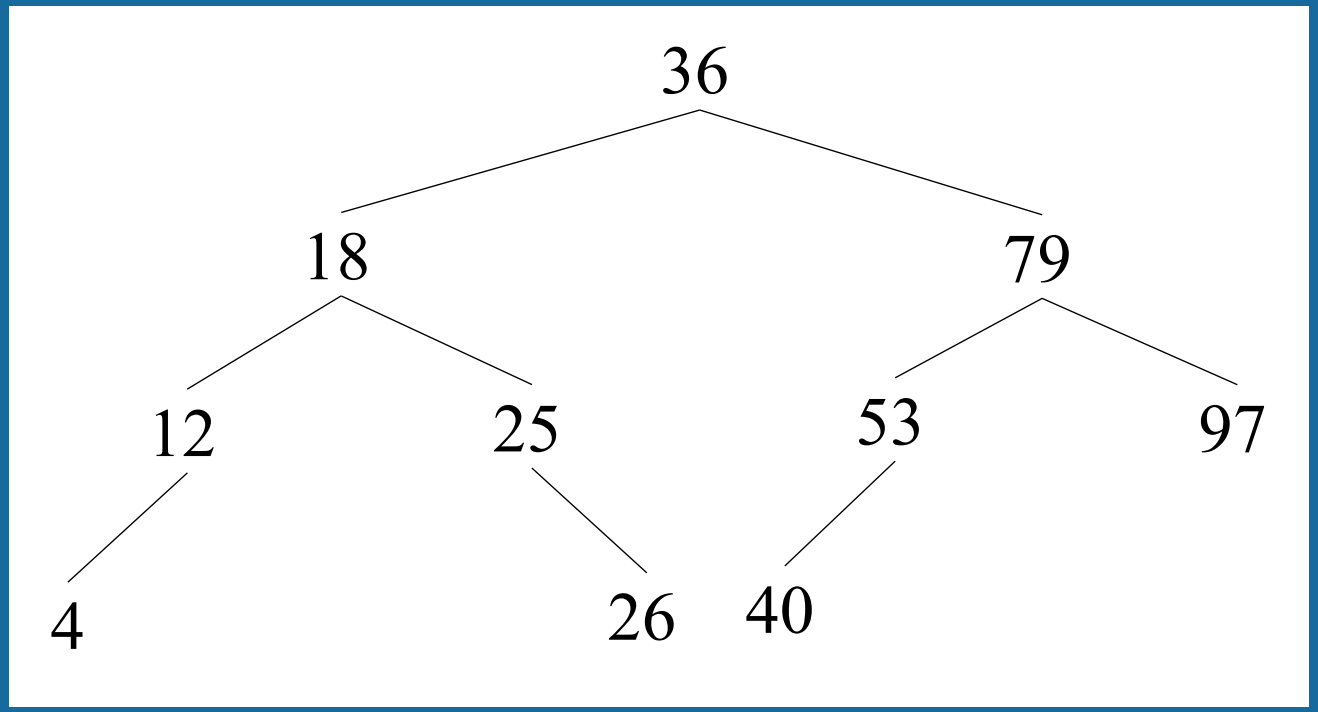
在平衡二叉树中插入一个结点后导致了不平衡。假设最低的不平衡的结点为A，在插入结点后A的平衡因子为-2，其右孩子的平衡因子为1，那么应做_____调整使其平衡。（一个结点的平衡因子定义为该结点的左子树深度减去右子树深度）

(A) LL　　(B) RR　　(C) LR　　(D) RL

27. 平衡二叉排序树插入结点

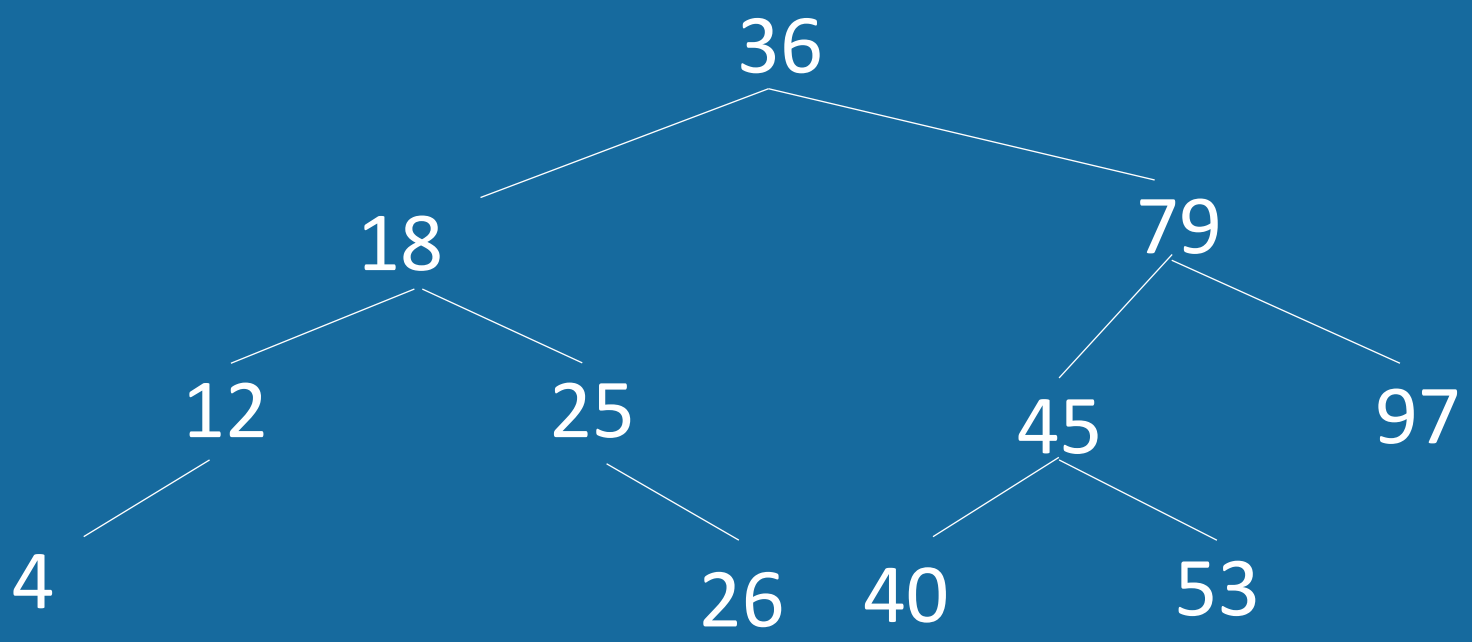
续1

若将关键字45插入如下平衡二叉排序树，试指出旋转类型(LL, LR, RR, RL)并画出旋转操作结束后的平衡二叉排序树。



27. 平衡二叉排序树插入结点

续2完



LR型

28. 基数排序各趟分配、收集的结果

对以下8个3位4进制数采用基数排序进行由小到大排序，写出对最低位、中间位以及最高位分别进行分配和收集的结果。

101, 002, 332, 220, 123, 301, 021, 231

第1趟：分配结果

0: 220

1: 101, 301, 021, 231

2: 002, 332

3: 123

收集结果

220, 101, 301, 021, 231, 002, 332, 123

28. 基数排序各趟分配、收集的结果

续1完

第2趟：分配结果 0: 101, 301, 002
1:
2: 220, 021, 123
3: 231, 332
收集结果： 101, 301, 002, 220, 021, 123, 231, 332
第3趟：分配结果 0: 002, 021
1: 101, 123
2: 220, 231
3: 301, 332
收集结果： 002, 021, 101, 123, 220, 231, 301, 332